



MASTER THESIS

Graph Neural Networks for Decentralized Multi-Agent Reinforcement Learning in Power-Grid Topology Control

Corentin Plumet

Supervisors

Prof. Fink

Prof. Peng

Dr. Nejjar

August 2026

Abstract

Power-grid topology control can relieve congestion by reconfiguring substation busbars, but fixed-width centralized actors scale poorly: their inputs and outputs grow with the grid and its combinatorial action space. Transmission systems are also organized into regions whose operators possess detailed local information and authority. Fully decentralized execution follows this structure, limits each actor to local observation and action spaces, and avoids a central coordinator. This thesis asks whether shared graph policies trained on the IEEE 14-bus system (**bus14**) can remain useful on the larger WCCI 2020 grid.

Using centralized-training, decentralized-execution MAPPO, we study sparse intervention, shared local graph encoders, and a grid-size-independent candidate-action scorer. We correct cross-grid input mismatch, use reduced WCCI action spaces, and compare zero-shot transfer, training from scratch, and fine-tuning. To separate the source of transfer, we distinguish encoder-only transfer, where the grid-specific action heads are replaced and relearned on WCCI, from complete-policy transfer, where the candidate-action actor is reused without changing its output interface.

On **bus14**, the strongest graph architectures exceed 99% mean survival, showing that the GNN actors can learn effective source-grid control. This does not imply that the same model family learns equally well when retrained on a larger grid. On difficult WCCI chronics, do nothing reaches 8.34% mean survival, while matched actors trained from scratch reach a median of only 9.6% at $k = 64$. In contrast, the architecture designed for weight transfer retains meaningful zero-shot performance: the best evaluated transferred graph actor, using the local safety gate, reaches 25.8%, about three times the do-nothing result. Transfer is therefore incomplete, but substantially more effective than retraining the same model family from scratch.

Contents

Abstract	i
1 Research Questions and Contributions	1
1.1 Thesis Title	1
1.2 Central Thesis Question	1
1.3 Research Questions	1
1.4 Contributions	2
2 Introduction	3
2.1 Topology control	3
2.2 Why topology control should be automated	3
2.3 Why use decentralized agents	4
2.4 Reinforcement learning for topology control	4
2.5 Starting point: MARL2Grid and MAPPO	5
2.6 Outline	5
3 Establishing a Baseline	6
3.1 Evaluation Protocol	6
3.2 IEEE 14-Bus Experiments	6
3.2.1 MLP Baseline	6
3.2.2 Initial Do-Nothing Bias	8
4 Sparse Intervention Control	10
4.1 Evaluation-Time Rho Heuristics	10
4.2 Intervention-Gated Actor	11
4.3 Fixed Sparse-Action Penalty	11
4.4 Adaptive Intervention Budget	12
4.5 Behavioral Cloning from Heuristics	12
4.6 Results	12
4.7 Preliminary WCCI Stress Test	14
5 Methods	15
5.1 Graph Representations	15
5.1.1 Busbar Representation	16
5.1.2 Disaggregated Representation	16
5.1.3 Disaggregated Line-Node Representation	17
5.1.4 Local actor information boundary and the global state graph	18
5.1.5 Graph actor and centralized critic	19
5.2 Substation Representation: Direct Edges and Summary Nodes	20
5.2.1 Direct same-substation edges (e)	20
5.2.2 Substation summary nodes (n)	21
5.3 Graph Readout	21
5.3.1 Mean and maximum graph readout	21
5.3.2 Virtual-node graph readout	22
5.4 Candidate-Action Scoring from the Disaggregated Line-Node Graph	23
5.4.1 From action indices to candidate scoring	23
5.4.2 What each action touches	24

5.4.3	Candidate pooling of the touched nodes	24
5.4.4	Scoring and the idle action	25
5.4.5	Global-maximum and action-delta extension	25
5.5	Cross-Grid Input Transformations	26
5.5.1	Physical scaling	26
5.5.2	Angle representation	26
5.6	Action-Space Reduction for Large Grids	26
6	Graph-Design Screening	28
6.1	Shared Protocol	28
6.1.1	Endpoint statistics	28
6.1.2	Compute	29
6.2	Screen A: Message-Passing Operator	29
6.3	Screen B: Encoder Depth and Width	29
6.4	Screen C: Graph Readout Scope and Reducer	31
6.5	Screen D: Structural Augmentations of the Busbar Graph	33
6.6	Screen E: Generator and Load Message Direction	34
6.7	Screen F: Candidate-Action Scoring	35
7	WCCI36: Target Environment and Controls	38
7.1	Target environment	38
7.1.1	Removing maintenance	38
7.1.2	Reduced action spaces	39
7.2	Evaluation protocol	39
7.3	Controls trained on WCCI	39
8	Cross-Grid Input Compatibility and Scaling	41
8.1	Measurement protocol	41
8.2	What the encoder reads	42
8.3	Results	42
8.4	Scale changes and offsets	43
8.5	Transforming the inputs	44
8.5.1	Running standardization does not align a new grid	44
8.5.2	Physical scaling for the power channels	44
8.5.3	Replacing absolute voltage angles	44
8.5.4	Results of the transformations	45
8.6	Inputs retained for transfer	45
9	Encoder-Only Transfer Across Grids	47
9.1	Question and hypotheses	47
9.2	Transferred components and controls	47
9.3	Results	48
9.3.1	Learning dynamics	48
9.3.2	Full-test comparison	49
10	Complete-Policy Transfer and Target Adaptation	51
10.1	Comparison rules	51
10.2	What controls zero-shot transfer?	51
10.3	Policy changes and the local heuristic	53
10.4	Does candidate-policy target training help?	55
10.5	Limits of the study	55
11	Discussion and Conclusions	57
11.1	Research questions revisited	57
11.2	What the experiments show together	58
11.3	Limitations and next steps	58
11.4	Conclusion	58

A	Additional Experimental Evidence	59
A.1	Sparse Intervention Control	59
A.1.1	Compact experimental details	59
A.1.2	Exact aggregate results	59
A.2	WCCI Transfer and Target-Control Results	60
A.2.1	Zero-Shot and Intervention Matrices	60
A.2.2	Target-trained controls	61
A.2.3	Per-chronic diagnostic	62
B	Graph-Policy Implementation Reference	63
B.1	Candidate-action scorer implementation	63
B.2	Exact graph inputs	63
B.2.1	Node inputs	64
B.2.2	Edge inputs	64
B.2.3	Deterministic physical scales	64
B.3	Graph construction and local boundaries	64
B.3.1	Candidate sizes and masking	64
B.3.2	Substation augmentations	65
B.3.3	Decentralized information boundary	65

Chapter 1

Research Questions and Contributions

1.1 Thesis Title

Graph Neural Networks for Decentralized Multi-Agent Reinforcement Learning in Power-Grid Topology Control

1.2 Central Thesis Question

Which graph architecture and action-selection design provide the most effective representation and control performance for decentralized policies, and to what extent can a policy trained on a small grid retain useful performance on a larger grid?

1.3 Research Questions

1. **Baseline learning:** Which training changes are needed for stable decentralized MAPPO topology control on `bus14`? Chapter 3 establishes the reference through the training, initialization, and initial-action-bias experiments.
2. **Sparse intervention:** Can the agents reduce unnecessary topology interventions without sacrificing source-grid survival? Chapter 4 compares evaluation-time heuristics, a learned intervention gate, action penalties, adaptive budgets, and behavioral cloning on both survival and intervention frequency.
3. **Graph representation:** Which graph representation, message-passing, graph-readout, and candidate-action design choices produce a useful local actor? Chapter 5 defines these choices and Chapter 6 evaluates them on `bus14`.

4. **Cross-grid transfer and adaptation:** To what extent do graph-actor weights learned on `bus14` retain useful performance on WCCI, and when zero-shot transfer is insufficient, does source initialization improve target-grid learning over training from scratch? Chapters 7–11 diagnose input compatibility, define the target protocol, test whether a frozen source encoder remains useful after replacing its action heads, and finally compare complete-policy zero-shot transfer with scratch training and fine-tuning before answering the combined research question.

1.4 Contributions

- **A stabilized decentralized MAPPO reference on `bus14`.** Chapter 3 isolates the training and initialization changes used by the later experiments.
- **A source-grid study of sparse intervention.** Chapter 4 evaluates several mechanisms under the joint criteria of survival and intervention frequency rather than survival alone.
- **A shared graph actor with size-independent candidate-action scoring.** Chapters 5 and 6 define the local graph, score actions from the nodes they affect, screen representation, message-passing, graph-readout, and action-scoring choices.
- **A diagnosis and correction of cross-grid input mismatch.** Chapter 8 measures what the shared encoder receives on both grids and motivates physically compatible power and angle inputs.
- **A controlled WCCI transfer and adaptation study.** Chapters 7–10 separate encoder-only transfer from complete-policy transfer, compare original and transformed input preprocessing, and evaluate zero-shot deployment, WCCI training from scratch, and source-initialized adaptation.

Chapter 2

Introduction

2.1 Topology control

Although a transmission grid is often represented as a fixed graph, operators can change its topology. Inside each substation, transmission lines, generators, and loads are connected to busbars. Moving an element from one busbar to another, or splitting a substation into two separate electrical nodes, changes the paths available to the power flow. The loading of the lines changes as a result. This type of reconfiguration is called topology control.

This flexibility is attractive because it can reduce congestion without adding new infrastructure. The usual alternatives are to redispatch generation, curtail renewable production, or build new transmission lines. Redispatch and curtailment cost money whenever they are used, while new lines are expensive and take years to build. Topology control instead uses switching equipment that is already installed, so the marginal cost of an action is close to zero [1]. This option becomes even more useful as renewable production increases, since variable generation makes power flows less predictable and pushes existing lines closer to their limits.

Its low operating cost does not make topology control easy to use. In practice, operators rely on a limited set of remedial actions prepared from experience and offline studies. Searching every possible configuration during operation is not realistic, which means that much of the available flexibility remains unused [2].

2.2 Why topology control should be automated

Two features of the problem make manual search difficult. The first is the number of possible actions. If a substation has d connected elements and two busbars, it already has 2^d possible assignments. Considering several substations together makes this number grow very quickly. Even the IEEE 14-bus system used in this thesis has 178 distinct unitary topology actions, while the 36-substation WCCI grid in Chapter 7 has more than 66 000. Its action space must therefore be reduced before training (Section 5.6).

The second difficulty is that these decisions are linked over time. An action taken now changes the grid state and may affect which contingencies can be handled later. Cooldown rules can also prevent another action at the same substation for several time steps. The operator must therefore choose quickly without considering only the immediate effect of an action.

Together, these properties make reinforcement learning (RL) a reasonable method to investigate. An RL agent learns through repeated interaction with a simulator, where each decision changes the next state and produces feedback. There is no simple formula for the best action, but the objective is clear: keep the grid operating and avoid a blackout. RTE created the Learning to Run a Power Network (L2RPN) competitions to study this problem. Grid2Op supports these experiments by providing the simulator, the load and generation time series, and the operating rules [3], [4].

2.3 Why use decentralized agents

Most published methods use one agent to control the full grid, but this is not how large power systems are usually operated. The network is divided into regions, and each control centre has detailed information about its own area and a less complete view of the rest of the system [2]. A decentralized model follows this organization more closely: each actor receives local observations, controls the substations in its region, and can operate without a central controller at execution time.

The same organization also helps with scalability. A central actor must process the full grid and produce scores for all available actions, so both its input and output grow with the network. In a multi-agent system, each actor chooses only among the actions of its own region. Adding another region therefore does not require every actor to represent the full joint action space.

Decentralization does not remove the difficulty of the problem; it changes its nature. Because all agents update their policies during training, the environment appears non-stationary from the point of view of any agent [5]. Local observations also hide part of the grid, so two actions that appear safe separately may be harmful when applied together.

Centralized training with decentralized execution helps address these issues. During training, a critic can use the global state to evaluate the joint behaviour of the agents, while each actor sees only the information that will be available in its region at deployment time [6], [7]. The critic is removed after training, leaving actors that operate independently. This is the setting used throughout the thesis.

2.4 Reinforcement learning for topology control

Previous RL work on topology control can be grouped into four main approaches. A broader review of graph-based RL for power systems is provided by Hassouna et al. [8].

Single-agent policies. The earliest approaches train one agent to control the whole grid. Subramanian et al. [9] showed that deep RL with a selected action set can outperform a do-nothing policy on L2RPN benchmarks. SMAAC, the winner of the L2RPN WCCI 2020 challenge, combines actor-critic learning over *afterstates* with a hierarchical decision about where and how to act [10]. These methods can perform well, but the action space must first be reduced, selected, or factorized.

Policies combined with heuristics. A second approach uses simple rules around a learned policy. A common choice is to query the agent only when the maximum line loading $\rho_{\max}$ becomes high, and to do nothing while the grid remains safe. PowRL combined this rule with an overload manager and a reduced action set to win the L2RPN NeurIPS 2020 robustness track [11].

Hierarchical and centrally coordinated multi-agent methods. A third family of methods divides the decision between several components while keeping a central coordinator. Manczak et al. [13] separate the decisions of when, where, and how to act. van der Sar et al. [14] use substation agents under a higher-level selector, while de Mol et al. [15] use a central agent to choose between regional proposals. These methods reduce the size of each decision, but a component still observes the full grid and coordinates the agents. They therefore do not show whether local policies can operate on their own.

Decentralized multi-agent control. Fully decentralized control has received less attention. Here, agents use local observations and act without communication or a central selector, which is the setting studied in this thesis. MARL2Grid addresses this gap by proposing a decentralized framework, because earlier research mainly considered single-agent control and did not represent the decentralized organization of grid operation [16].

Graph representations. Regardless of the control structure, the representation of the grid matters. A flat vector hides the network structure and has a size tied to one grid. A graph neural network (GNN) follows the grid connections and can use the same parameters on networks of different sizes [17], [18], [19].

However, a homogeneous graph may omit connections that a topology action could create. Heterogeneous graphs can reduce this *busbar information asymmetry* [20]. Chapters 5 and 6 compare these choices.

2.5 Starting point: MARL2Grid and MAPPO

This work builds on MARL2Grid [16], the multi-agent extension of RL2Grid [21]. MARL2Grid connects Grid2Op to a multi-agent training pipeline and defines the regions, observations, action spaces, rewards, and training loop used as the starting point of this thesis.

Grid partition. The grid is divided in advance into fixed, non-overlapping regions, with one agent assigned to each region. In the IEEE 14-bus environment (**bus14**), the three agents control substations $\{0, 1, 2, 4\}$, $\{3, 6, 7, 8\}$, and $\{5, 9, \dots, 13\}$. These regions contain 52, 50, and 76 non-idle actions, which together form the 178 unitary actions of the full grid. Each agent observes its own region and chooses an action at one of its substations. The agents then act at the same time without seeing each other’s choices, and action 0 always means do nothing.

Reward and evaluation metric. The reward is mainly based on survival, with +1 given for each survived step and an additional term that penalizes overloads. Training therefore encourages the agents to keep the episode alive. For one chronic, survival is defined as the number of survived time steps divided by the maximum episode length, and mean survival is this value averaged over the evaluated chronics.

MAPPO. Multi-agent PPO (MAPPO) [7] extends PPO [22] to centralized training with decentralized execution. Each actor maps its local observation to a probability distribution over its local actions. During training, PPO limits the size of each policy update so that the actors do not change too abruptly. A centralized value function can use the global state to evaluate the agents and estimate the advantage of their actions. This global information is never given to the actors, which continue to use only their local observations. Once training is complete, the centralized value function is no longer needed and deployment remains decentralized.

Initial baseline. The initial model uses flat multilayer-perceptron actors and one critic trained on **bus14**. Without further tuning, this baseline is unstable and has low survival. It is also tied to one grid because the actor input size depends on the number of substations, so its weights cannot be used directly on a larger network. The next chapters address these limitations in sequence: they first stabilize training, then reduce unnecessary interventions, and finally replace the fixed-size actor with a graph-based model.

2.6 Outline

The thesis is organized in two stages. Chapters 3 and 4 first focus on learning a reliable decentralized controller on **bus14**. Chapter 3 establishes a stable flat MAPPO baseline, and Chapter 4 studies how to reduce unnecessary interventions. Its preliminary WCCI stress test also reveals the limitations of applying a fixed-size actor to a larger grid.

Chapters 5–10 then develop and evaluate cross-grid transfer. Chapter 5 introduces the graph actor and candidate-action scorer, and Chapter 6 evaluates their main design choices on **bus14**. Chapter 8 makes the physical inputs more comparable across grids, while Chapter 7 defines the WCCI environment and evaluation protocol. Chapter 9 tests whether the encoders selected by Screens A–E remain useful when their grid-specific action heads are replaced. Chapter 10 then studies the stricter case in which the complete candidate-action policy is transferred, with and without target adaptation. Chapter 11 closes the thesis by answering the research questions and discussing the limits of the evidence.

Chapter 3

Establishing a Baseline

This chapter builds a solid MLP actor baseline for the IEEE 14-bus task.

3.1 Evaluation Protocol

All experiments use an 80/20 train/test chronic split. The 80% training split is used for policy updates, while the 20% test split measures how well the policy generalizes. During training, checkpoint evaluation uses 20 chronics from the training split. Each evaluation is a new rollout, so these 20 chronics change from one evaluation to the next rather than forming a fixed subset. The checkpoint-selection score is mean survival over the 20 chronics in the current rollout.

For one chronic, survival is the number of survived time steps divided by that chronic’s maximum episode length. Mean survival is the unweighted mean of these normalized durations over the evaluated chronics. Results are reported as mean curves across seeds, with shaded regions showing the spread.

3.2 IEEE 14-Bus Experiments

The initial MAPPO baseline from the MARL2Grid framework led to unstable policies and low mean survival on the IEEE 14-bus topology-control task. The experiments below test changes to the training and to the architecture, in order to build a strong baseline.

3.2.1 MLP Baseline

Question. Which changes are needed to get a stable MLP MAPPO baseline on the 14-bus task?

Baseline and changes. We start from the MLP MAPPO baseline of the MARL2Grid framework and test several main changes meant to stabilize training:

- using reward normalization;
- changing the critic update so that the critic is updated once for the whole batch instead of once per agent;
- tuning the optimization hyperparameters to stabilize training, as summarized in Table [3.1](#).

Table 3.1: New hyperparameters for MAPPO training.

Hyperparameter	New Value	Paper Baseline
ENVIRONMENT		
n_envs (number parallel environment)	20	10
PPO OPTIMIZATION		
Total time step	15,000,000	25,000,000
Actor learning rate	1e-4	3e-5
Critic learning rate	1e-4	3e-5
Gamma	0.9	0.99
Update epochs	10	80
Minibatches	16	4
Maximum gradient norm	1.0	10.0

Evidence. Table 3.3 reports final and peak mean survival for the core ablations. Figure 3.1 overlays all core MLP baseline curves.

Interpretation. The **Optimized MLP Baseline** is retained because it gives the best overall balance of high survival, fast convergence, and stable training. Some ablations reach comparable or slightly higher isolated endpoints, but their trajectories are slower or more variable. The selection therefore reflects stability and performance across training rather than the final point alone.

Table 3.2: MLP baseline run configurations. The reference run is **Optimized MLP Baseline**.

Run family	Actor/critic hidden layers	Reward norm.	Initial action-0 prob.	Optimized critic update	Difference from Optimized MLP Baseline
Optimized MLP Baseline	$3 \times 256 / 3 \times 256$	true	0.0	true	Reference baseline
Disable optimized critic update	$3 \times 256 / 3 \times 256$	true	0.0	false	Disables optimized critic updates
Smaller MLP actor/critic	$2 \times 256 / 2 \times 256$	true	0.0	true	Uses a smaller MLP actor and critic
Disable reward normalization	$3 \times 256 / 3 \times 256$	false	0.0	true	Disables reward normalization
Initial Bias 50%	$3 \times 256 / 3 \times 256$	true	0.5	true	Uses action-0 bias 0.5
Initial Bias 70%	$3 \times 256 / 3 \times 256$	true	0.7	true	Uses action-0 bias 0.7

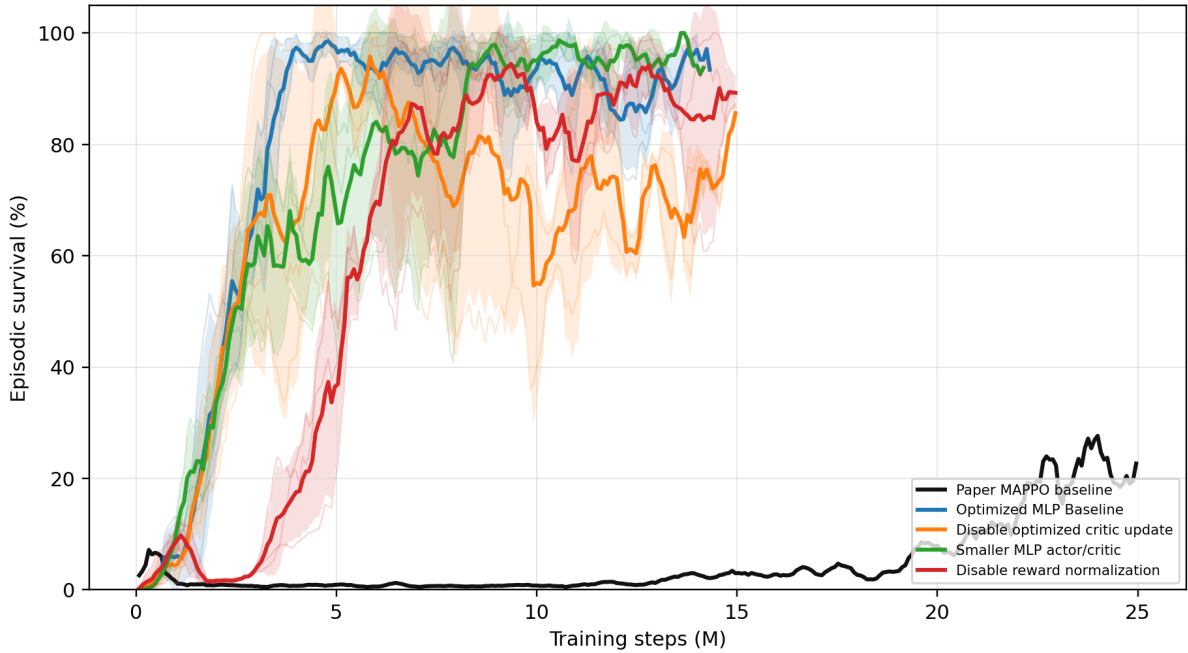


Figure 3.1: All MLP baseline core rerun survival curves shown on one axis.

Table 3.3: MLP baseline core rerun mean-survival summary.

Run	Intended change	Final mean survival (%)	Peak mean survival (%)
Optimized MLP Baseline	Optimized MLP baseline	93.4	98.6
Disable optimized critic update	Non-optimized critic update	93.1	95.8
Smaller MLP actor/critic	Smaller MLP architecture	93.7	100.0
Disable reward normalization	No reward normalization	81.8	97.1

3.2.2 Initial Do-Nothing Bias

Question. Does biasing the initial policy toward action 0, the do-nothing action, help or hurt learning? The idea makes sense: under normal conditions, human operators usually leave the grid alone and take no action.

Baseline and changes. Our baseline uses no initial do-nothing bias in this rerun set. The bias controls keep the same general protocol but change the initial probability mass assigned to action 0.

Implementation detail. The bias is applied only at initialization: it changes the starting action distribution before learning, without changing the reward, architecture, or PPO objective. It is implemented by raising the initial pre-softmax logits of the do-nothing action, so that the policy picks this safe action more often before any learning update happens. This makes the ablation a test of exploration bias rather than a test of model capacity.

Evidence. Table 3.4 and Figure 3.2 compare the zero-bias baseline with Initial Bias 50% and Initial Bias 70%.

Interpretation. The zero-bias model is retained because it reaches high survival earlier and more consistently. Although the 70% bias finishes higher, it learns slowly, while the 50% bias remains unstable. Neither gives a better overall stability–performance trade-off.

Table 3.4: Initial do-nothing bias mean-survival summary.

Run	Final mean survival (%)	Peak mean survival (%)
Optimized MLP Baseline	93.4	98.6
Initial Bias 50%	50.8	70.2
Initial Bias 70%	99.3	100.0

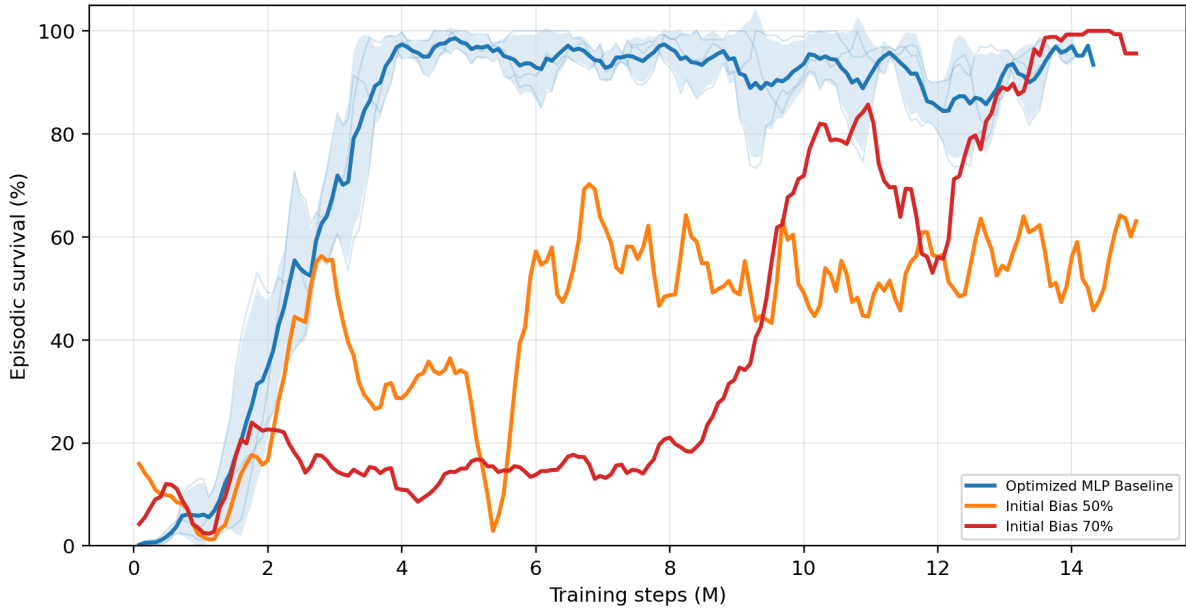


Figure 3.2: Optimized MLP baseline compared with initial do-nothing bias controls **Initial Bias 50%** and **Initial Bias 70%**.

Chapter Summary

Purpose: Establish a stable, high-performing flat MAPPO reference on `bus14`.

Content: Compare the original training recipe with reward normalization, critic-update, model-size, and initial action-0-bias ablations.

Findings:

- **MLP optimization:** The selected reference enters the high-survival regime earlier and sustains it more consistently than the main training ablations. Its final value is about 93%, but it is not the largest isolated endpoint in the screen.
- **Do-nothing initialization:** A 50% bias is weak and unstable, whereas a 70% bias learns slowly before reaching a high final value. The effect therefore depends strongly on the bias level.

Takeaway: The tuned zero-bias actor is used as a stable reference because of its convergence trajectory, not because it wins every endpoint comparison. Chapter 4 next asks whether this actor can intervene less often without losing survival.

Chapter 4

Sparse Intervention Control

The previous chapter optimized survival on held-out chronics. This chapter adds a second objective: sparsity. A policy that keeps the grid alive by reconfiguring substations at every opportunity is impractical for a control room. Interventions carry operational costs, and an agent acting constantly is not trustworthy. The question is whether decentralized agents can keep high survival while intervening rarely, locally, and only when necessary.

In the literature, sparse activity is most often enforced by hard-coding a physical threshold that decides when the agent is allowed to act. Instead of relying on an external override, this chapter tries to make sparse intervention something the agent learns by itself.

Mechanisms are therefore measured on two competing axes: mean survival and intervention sparsity. A good mechanism must improve sparsity without hurting survival.

Question. Can the agents keep high survival while behaving more like operators: acting rarely and mainly when the grid state makes an intervention useful?

Baseline and change. This block starts from the flat decentralized MAPPO topology-control actor. For agent i , the baseline policy is one categorical distribution over the full local action space:

$$\pi_i(a_i \mid o_i), \quad a_i \in \{0, 1, \dots, |\mathcal{A}_i| - 1\}.$$

Action 0 is the no-op action. During training, actions are sampled from the policy, while deterministic evaluation uses the greedy action by default:

$$a_{i,t}^{eval} = \arg \max_a \pi_i(a \mid o_{i,t}).$$

This chapter tests five ways to reduce unnecessary topology changes: evaluation-time rho overrides, an intervention-gated actor, fixed non-idle action penalties, adaptive intervention budgets, and behavioral cloning from heuristics.

The intervention gate separates whether to act from which action to take, following the hierarchical splits used in topology control [13], [14]. The fixed and adaptive penalties instead treat non-idle interventions as a limited resource, following constrained and budgeted reinforcement learning [23], [24], [25].

4.1 Evaluation-Time Rho Heuristics

Implementation detail. The rho heuristic is an evaluation-only diagnostic. It does not change the training objective and it does not prove that the policy learned sparse behavior. The policy first proposes the deterministic action $\bar{a}_{i,t}$, then the evaluator may replace it with action 0.

The global version uses the pre-action maximum line loading

$$\rho_t^{global} = \max_{\ell \in \mathcal{L}} \rho_{\ell,t},$$

and executes

$$a_{i,t}^{exec} = \begin{cases} 0, & \rho_t^{global} < \rho_{safe}, \\ \bar{a}_{i,t}, & \text{otherwise,} \end{cases} \quad \rho_{safe} = 0.90.$$

The local version is decentralized. Agent i computes the maximum rho on the lines touching its regional substations,

$$\rho_{i,t}^{local} = \max_{\ell \in \mathcal{L}_i} \rho_{\ell,t},$$

and only that agent is forced to no-op when $\rho_{i,t}^{local} < \rho_{safe}$. A boundary line can appear in the local neighborhood of both adjacent agents, but no communication is required: both agents can observe the line through their own physical neighborhood.

Interpretation. The heuristic is useful to ask what happens if safe-state interventions are blocked, and if succesful, it can prove that the agents act too often.

4.2 Intervention-Gated Actor

Implementation detail. The gated actor changes the actor factorization. Instead of one categorical head over all actions, each agent has a gate head and a non-idle action head:

$$\pi_i^{gate}(g_i | o_i), \quad g_i \in \{0, 1\},$$

where 0 means do nothing and 1 means intervene, and

$$\pi_i^{nonidle}(b_i | o_i), \quad b_i \in \{1, \dots, |\mathcal{A}_i| - 1\}.$$

During training, the gate is sampled first. If $g_{i,t} = 0$, then $a_{i,t} = 0$. If $g_{i,t} = 1$, the final action is sampled from the non-idle head. The PPO log-probability of the executed action is therefore

$$\log P(a_i = 0) = \log P(g_i = 0),$$

and, for $a_i > 0$,

$$\log P(a_i) = \log P(g_i = 1) + \log P(b_i = a_i | g_i = 1).$$

Training always uses the hierarchical sampling procedure above. At deterministic evaluation, the resulting policy can be decoded in two ways. The `hierarchical_greedy` rule first selects the most likely gate decision and, if it chooses intervention, the most likely non-idle action. The `final_action_map` rule instead selects the most likely executed action using the final probabilities defined above. MAP is therefore not a separate training mode; it only replaces sampling with a deterministic decision. It can favor action 0 because the total intervention probability is divided among several non-idle actions. The entropy variant used by the gate-plus-budget diagnostic is specified in [Appendix A.1.1](#).

Interpretation. The gate is learned and decentralized, but it can limit exploration more than a reward penalty does. If it collapses early to no-op, the non-idle head is rarely used and gets little learning signal.

4.3 Fixed Sparse-Action Penalty

Implementation detail. The fixed-penalty experiments compare several penalty strengths. In each run, a reward penalty is applied to the final executed action of each agent:

$$r'_{i,t} = r_{i,t} - \lambda_{fixed} \mathbf{1}[a_{i,t} \neq 0],$$

with $\lambda_{fixed} \in \{0.0, 0.001, 0.003, 0.01\}$. The complete 24-run actor, penalty, and seed grid is reported in [Appendix A.1.1](#).

Interpretation. This mechanism changes only the training reward. It does not override actions during evaluation and it does not block exploration during training. The policy can still sample non-idle actions, but each such action must be useful enough to pay its fixed cost.

4.4 Adaptive Intervention Budget

Implementation detail. The adaptive intervention budget does not use a fixed intervention penalty. Instead, each agent has its own penalty coefficient λ_i , which is adjusted after every rollout. The local-safe version used in Figure 4.1 first converts the maximum line loading visible to agent i into a safety weight:

$$w_i^{local}(s_t) = \sigma(k(\rho_{safe} - \rho_{i,t}^{local})).$$

When the local grid is safely below $\rho_{safe} = 0.90$, this weight is high. It decreases as the local lines approach or exceed the threshold. The action cost is then

$$c_i(s_t, a_{i,t}) = \mathbf{1}[a_{i,t} \neq 0]w_i^{local}(s_t).$$

Doing nothing always has zero cost. An intervention is expensive in a safe state but remains relatively cheap when the local grid is stressed and an action may be necessary. During the rollout, this cost is subtracted from the original reward:

$$r'_{i,t} = r_{i,t} - \lambda_i c_i(s_t, a_{i,t}).$$

A large value of λ_i therefore makes interventions more expensive, whereas a small value leaves the policy freer to act.

At the end of the rollout, the average observed cost is

$$\hat{C}_i = \frac{1}{T} \sum_{t=1}^T c_i(s_t, a_{i,t}).$$

This value is compared with the target budget d , and the penalty coefficient is updated as

$$\lambda_i \leftarrow \text{clip}(\lambda_i + \eta(\hat{C}_i - d), 0, \lambda_{\max}).$$

If $\hat{C}_i > d$, the agent has exceeded its budget, so λ_i increases and future interventions become more expensive. If $\hat{C}_i < d$, the penalty decreases. The budget d is therefore a target used to adjust the penalty after each rollout; it is not a fixed amount subtracted from every reward. The value $\rho_{safe} = 0.90$ is only the center of the smooth safety weight; it does not force the agent to do nothing. The multiplier hyperparameters and complete budget ablation are reported in Appendix A.1.1.

4.5 Behavioral Cloning from Heuristics

A teacher-student diagnostic tests whether the behavior of the local rho heuristic can be copied into the policy weights. The student is trained by behavioral cloning and evaluated without the heuristic. The training setup and exact results are reported in Appendix A.1.1.

4.6 Results

Figure 4.1 summarizes mean survival and the mean action-0 fraction across agents. Exact condition means are reported in Appendix A.1.

Table 4.1: Sparse-control mechanisms and where they affect the policy.

Method	Training effect	Evaluation effect	Main risk
Baseline	Samples from the flat policy	Greedy argmax by default	No explicit mechanism to favor sparse control
Rho heuristic	No training change	Hard no-op override in safe states	Evaluation is manually changed
Intervention gate	Changes action factorization and sampling	Final-action MAP or hierarchical greedy	Gate collapse can starve non-idle exploration
Fixed penalty	Fixed reward penalty on non-idle actions	No action override	Penalty coefficient must be tuned
AIB local-safe	Adaptive state-dependent reward penalty	No action override	Depends on the local rho
Behavioral cloning	Distills heuristic into actor weights	No action override	Safety-weight design Struggles with extreme action-0 class imbalance

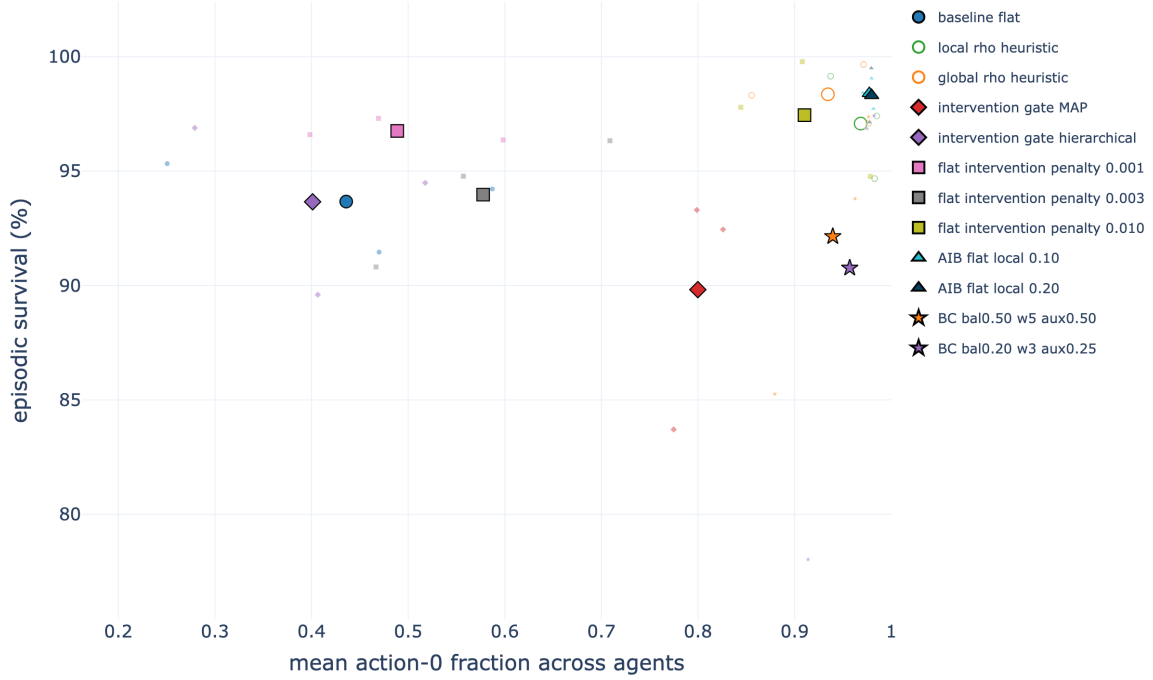


Figure 4.1: Joint action-0/mean-survival comparison of the selected sparse-control mechanisms on **bus14**. Large markers show condition means and faint markers the individual seeds. The plot uses checkpoint-aligned action-0 rates and full-test mean survival, so it shows the chapter’s two objectives at once.

Interpretation. Local-safe AIB is the clear winner among the learned methods. For $d = 0.10$ and $d = 0.20$, it raises mean survival from the baseline’s 93.67% to about 98.4%, while reducing the mean non-idle action fraction from 56.6% to about 2.1–2.2%. It therefore improves both objectives at the same time instead of trading survival for sparsity. This result suggests that the adaptive local-safe cost helps each agent learn to reserve interventions for states in which they are useful, while remaining fully decentralized.

4.7 Preliminary WCCI Stress Test

As an early scale test, representative mechanisms were retrained from scratch with a fixed-width MLP actor on the maintenance-enabled WCCI grid. The mechanisms changed intervention frequency, but none reliably improved survival over the weak baseline, and the differences were small relative to seed variation. Because this pilot changes the grid, action space, and maintenance setting at the same time, it cannot isolate the cause. It nevertheless motivates the size-independent graph actor introduced next.

Chapter Summary

Purpose: Keep survival while reducing unnecessary topology interventions.

Content: Compare evaluation-time rho overrides, a learned intervention gate, fixed action penalties, adaptive intervention budgets, and behavioral cloning from a sparse teacher.

Findings:

- **Evaluation heuristics** reach 97–98% mean survival with action 0 in 94–97% of decisions, but impose sparsity after the policy acts.
- **A fixed penalty** already gives a strong learned controller: $\lambda_{fixed} = 0.010$ reaches 97.44% mean survival and a 0.905 action-0 fraction without an evaluation override.
- **Local-safe AIB** gives the best observed learned trade-off in this experiment: budgets $d = 0.10$ and $d = 0.20$ reach about 98.4% mean survival and 0.98 action-0 usage. The gated variants are less reliable and can collapse to almost permanent no-op behavior.

Takeaway: Sparse behavior can be learned on **bus14**, but the preliminary maintenance-enabled WCCI stress test does not isolate transfer and its fixed-width MLP policies remain weak. This motivates the graph representation introduced next.

Chapter 5

Methods

Chapter 4 made the flat policy behave more like an operator on `bus14`, but the first WCCI test was still weak. The sparse control mechanisms changed when the policy acted, but they did not solve a more basic problem: the MLP input and output sizes were tied to one grid. This chapter replaces that fixed representation with graph-based actors that can be used on grids of different sizes. It introduces the choices studied later in the thesis: graph construction, substation augmentations, graph readout, candidate-action scoring, cross-grid input transformations, and action-space reduction.

5.1 Graph Representations

I use three graph representations. They mainly differ in which physical objects receive their own node. The *busbar* graph places all measurements on busbar nodes. The *disaggregated* graph adds separate generator and load nodes, but still represents transmission lines as edges. The *disaggregated line-node* graph also turns each transmission line into a node.

Table 5.1 compares the three choices. The representation also changes what a local actor can see at the far end of a boundary line. The graphs therefore differ not only in size, but also in the information they contain. Section 5.1.4 describes this boundary in detail.

Table 5.1: The three graph schemas. Each row follows from how much equipment receives its own node. In the node counts, S is the number of substations, B the busbars per substation, N_g and N_d the included generators and loads, and L the included transmission lines.

Aspect	busbar	disaggregated	disaggregated line-node
gnn_graph_type	bus	heterogeneous	heterogeneous_line
Node types	Busbar	Busbar, generator, load	Busbar, generator, load, transmission line
Generator and load state	Summed and averaged onto the busbar	One node per asset	One node per asset
Transmission line	Direct busbar-to-busbar edge	Typed direct busbar-to-busbar edge	Typed node between its endpoint busbars
Line state	Continuous edge attributes	Continuous edge attributes	Features of the line node
Shortest message path	One step between adjacent busbars	One step between adjacent busbars; two from an asset to a remote busbar	Two steps between adjacent busbars through a line node
Hidden line state	None: the edge modifies a message	None: the edge modifies a message	Yes: each line obtains and updates an embedding
Far-end context in a local actor graph	Live neighboring busbar, including aggregated power and angle	Live neighboring busbar, but no neighboring generator or load nodes	None; the shared line node replaces the need for a far-end busbar
Node count	SB	$SB + N_g + N_d$	$SB + N_g + N_d + L$

The message-path row describes distances in each graph schema. It does not set the encoder depth K , which is selected experimentally in Chapter 6.

5.1.1 Busbar Representation

The busbar graph is the simplest and most compact representation. Busbars are the only nodes and active transmission lines are the edges. Generator and load measurements are added to the busbar to which each asset is currently connected.

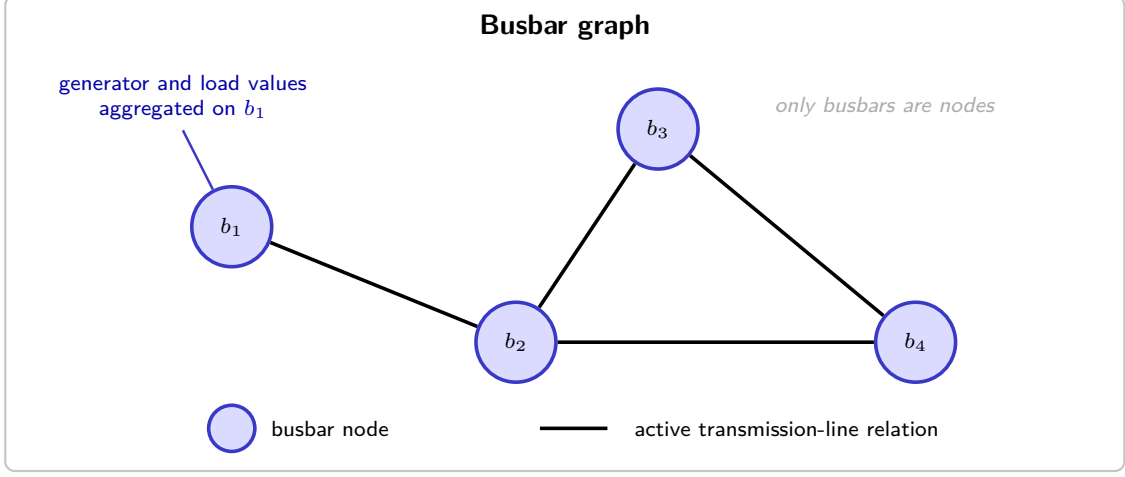


Figure 5.1: Busbar graph representation. Blue nodes are busbars and black connections are active transmission-line relations. Generator and load quantities are aggregated into the features of their associated busbars.

Nodes and features

For S substations and at most B busbars per substation, the graph contains one node $i(s, b)$ for every substation–busbar pair. Generator and load powers are summed on their current busbar, their angles are averaged, and the substation cooldown is copied to both busbars. The complete feature inventory is reported in Appendix B.2.1.

Relations and edge features

The only physical relation in this graph is a transmission line between two busbars. Each active line edge carries its status, loading, overload duration, and cooldown, together with maintenance timing when that information is available. Appendix B.2.2 gives the exact edge vector.

5.1.2 Disaggregated Representation

The disaggregated graph keeps the busbars as its backbone, but gives each generator and load a separate node. Directed or bideirectionnal edges connect these assets to their current busbar. Transmission lines remain direct edges between busbars.

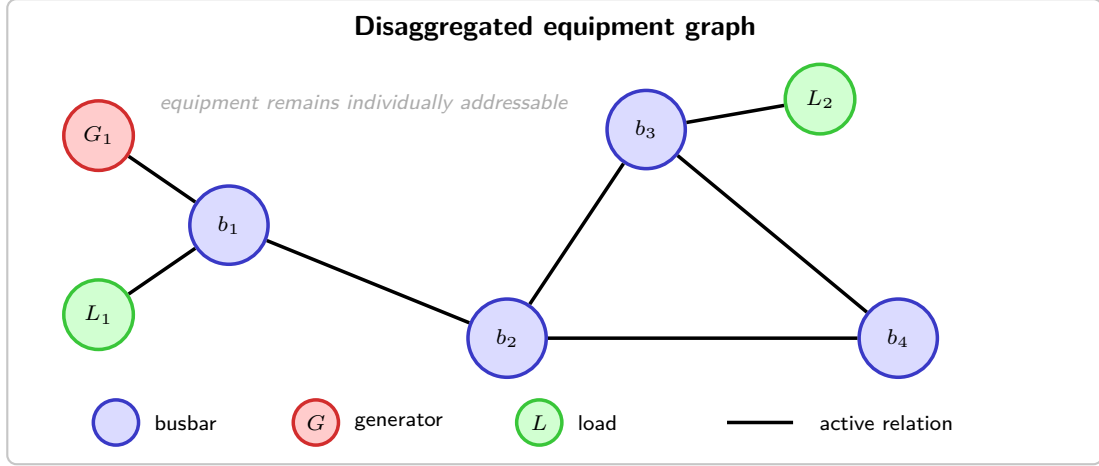


Figure 5.2: Disaggregated representation. Busbars are blue, generators are red, and loads are green. Physical transmission lines remain direct relations between busbar nodes.

Nodes and features

The busbar nodes are the same as in Section 5.1.1. The graph then adds one node for every generator and load whose substation belongs to the local graph. All three node types use one common feature layout so that they can pass through the same encoder. Type indicators identify busbars, generators, and loads; physical values occupy only the columns relevant to that type. Disconnected assets remain indexed but their power and angle are zero. The ordered vector and per-type inventory are given in Appendix B.2.1.

Relations and edge features

Every directed edge uses a common layout containing a physical line block and a one-hot relation block. Transmission-line edges populate both; asset attachments and structural edges use a zero physical block and identify only their type and direction. This prevents a structural relation from looking like a measured line. The exact columns and activation rules are listed in Appendix B.2.2.

The direction of generator and load edges can be chosen independently. Each asset type can use `bidirectional`, `asset_to_busbar`, or `busbar_to_asset`. The default is `bidirectional`. Transmission-line and same-substation relations remain `bidirectional`.

5.1.3 Disaggregated Line-Node Representation

The disaggregated line-node graph goes one step further. Instead of a direct edge between two busbars, each transmission line becomes a node placed between them. Busbars, generators, loads, and lines are therefore all explicit nodes.

Nodes and features

All four node types again share one ordered feature layout. Transmission-line state now occupies the line-node row, while generator and load rows carry their own power and angle. Optional maintenance values are included only when available. Appendix B.2.1 reports the exact columns, dimensions, and zero-filled entries.

Relations and edge features

The edges now describe only the type and direction of each connection. They need no continuous line attributes because those values already belong to the line node. Every active edge therefore selects exactly one relation type, while inactive candidates are zero-filled and removed before message passing. The nine relation columns and activation rules are listed in Appendix B.2.2.

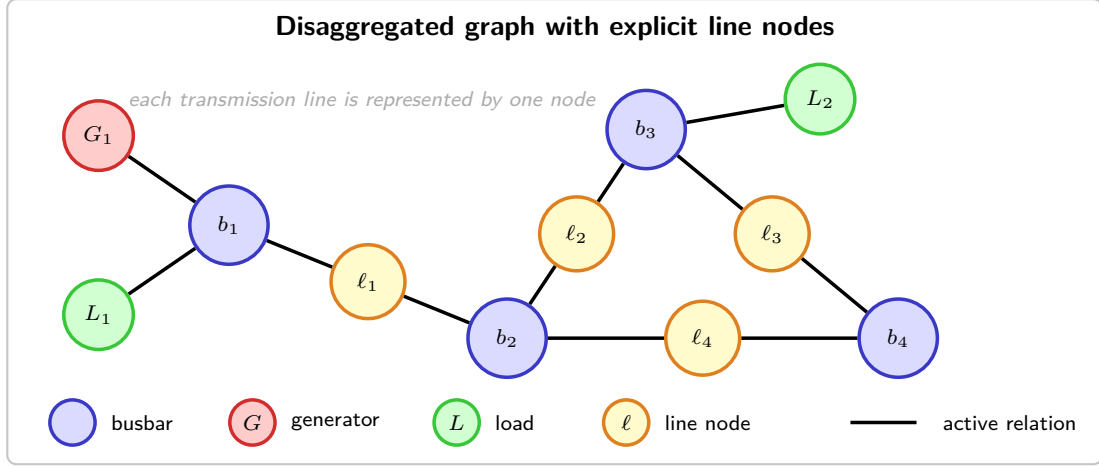


Figure 5.3: Disaggregated line-node representation. Yellow nodes represent transmission lines and connect the blue busbars at their endpoints; generators and loads remain separate terminal nodes.

5.1.4 Local actor information boundary and the global state graph

Agent a controls a set of substations $\mathcal{C}_a$. A boundary line links this region to a substation controlled by another agent. The local graph includes only the outside information needed to represent that shared line, and the exact amount depends on the graph representation.

Why one hop, and why not more. I use a one-hop boundary to keep the actor local while the critic still observes the full grid during training. Power flow is not truly limited to one hop, so this is a practical choice rather than a claim of optimality. The loading and angle difference of each boundary line provide the most direct interaction with the neighboring region without revealing all of its private asset states.

Visibility and masking

A neighboring busbar is visible only while an active boundary line connects it to the controlled region. If that connection disappears, the node features are set to zero, its edges are disabled, and the node is removed from the graph readout. The node therefore contributes neither to message passing nor to the graph embedding. A same-substation structural edge cannot make it visible again.

Controlled nodes remain visible even when they are disconnected, because the agent may reconnect equipment to them. In the line-node graph, a shared line node also remains present, but its attachment edges are disabled while the line is offline. An attachment edge is active only when the line is connected and the current busbar assignment matches that edge. Nodes for neighboring generators and loads are never created.

Representation-dependent information

The three representations expose different views of the neighboring region. Figure 5.4 makes this difference explicit; the corresponding inventory is retained in Appendix Table B.6.

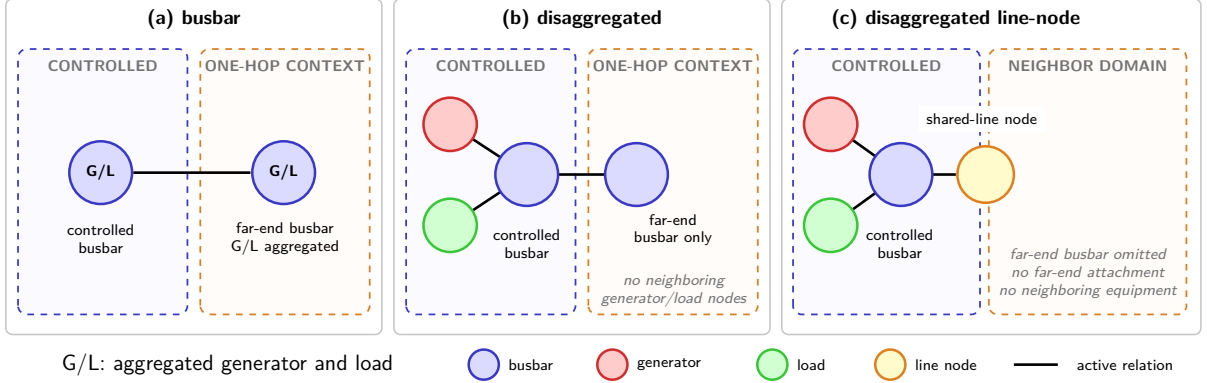


Figure 5.4: One live boundary line, seen under each representation. In every panel the blue dashed region is the agent’s controlled domain and the orange dashed region is what lies beyond it. The busbar graph (a) reaches a far-end busbar carrying the neighbor’s generation and load aggregated onto it. The disaggregated graph (b) reaches the same busbar, but no neighboring generator or load node is ever constructed, so that busbar carries no measurement of its own. The disaggregated line-node graph (c) reaches only the shared line node, and no far-end busbar exists at all.

In the busbar and disaggregated graphs, the far-end busbar is needed because the boundary line is an edge and an edge must have two endpoints. Only the busbar used by the active line is visible; the other candidate busbars remain masked. If the line disconnects, no far-end busbar remains visible. In the disaggregated graph, this neighboring busbar contains only type and domain indicators because the power and angle measurements belong to generator and load nodes, which are not created outside the controlled region. The boundary line edge is therefore the only source of neighboring measurements. The line-node graph removes the far-end busbar completely and stores the shared measurements on the line node. When the line disconnects, the line node remains with `line_status`= 0, while its attachment edges become inactive.

These information limits apply only to the actors. During training, the centralized critic still receives the normalized flat state of the full grid. At execution time, each actor uses only its own local graph. This is the centralized-training, decentralized-execution setting used by MAPPO [7].

5.1.5 Graph actor and centralized critic

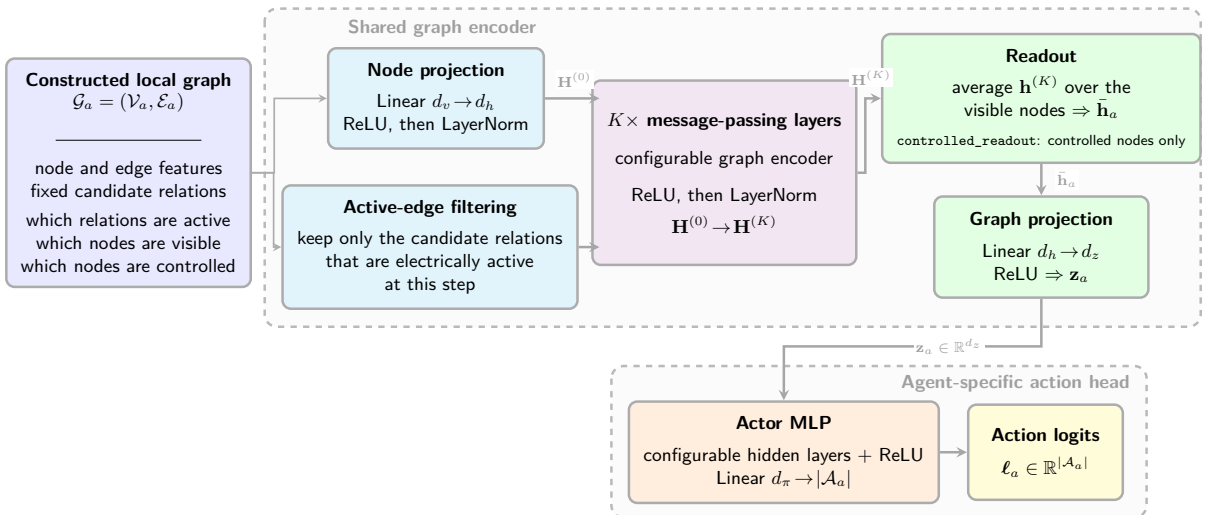


Figure 5.5: End-to-end graph-actor architecture. Candidate edges masked by the current topology are removed before message passing. The graph encoder and its output projection are shared across agents, while the final MLP has one output per action available to agent a and is therefore agent-specific.

Figure 5.5 shows the complete actor. The node features are first projected to a hidden width d_h . The encoder then applies K message-passing layers before reducing the node states to one graph embedding of width d_z . The encoder family, depth K , and hidden width d_h are experimental choices and are therefore not fixed in this chapter. They are selected in Chapter 6. The encoder and output projection are shared across agents, but the final action heads are separate because agents can have different numbers of actions.

The critic is a separate MLP that receives the normalized flat state. It does not reuse the actor’s graph embedding and is present only during training. Its normalization is independent of the graph-input transformation. The appendix records the exact encoder, graph-readout, output-projection, and action-head equations used by the evaluated configurations.

5.2 Substation Representation: Direct Edges and Summary Nodes

The base graphs do not directly connect the two busbars of one substation. I test two optional ways to add this internal communication. Factor e creates direct edges between the busbars. Factor n creates a learned summary node for the substation. In both cases, the added nodes and edges pass through the same projection, masking, and message-passing pipeline as the rest of the graph.

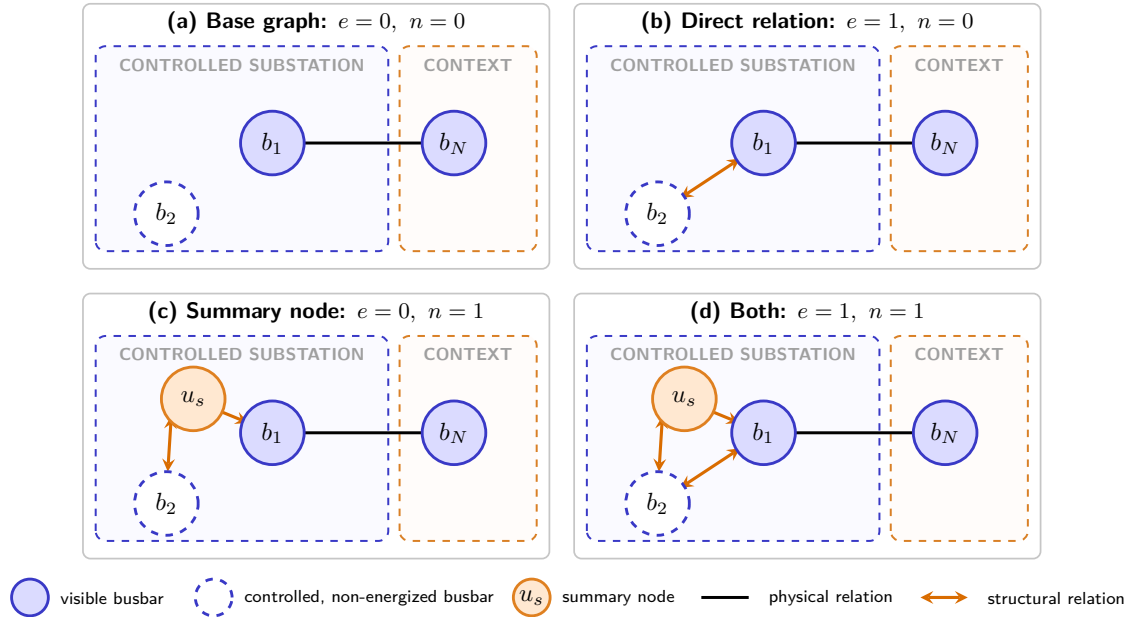


Figure 5.6: The four combinations of the independent substation factors e and n . The hollow node b_2 is controlled but non-energized. Factor e adds a bidirectional relation between the controlled busbars, while factor n connects a summary node u_s to both. Neither augmentation extends to the contextual busbar b_N , which retains only its physical relation.

The placement of these augmentations in all three graph schemas, together with their exact feature values, is documented in Appendix B.3.2.

5.2.1 Direct same-substation edges (e)

Factor e adds the edge $i(s, b) \rightarrow i(s, b')$ for every ordered pair of different busbars in the same controlled substation. Both directions are present and remain active throughout the episode. These are structural edges, not electrical lines. A controlled busbar can therefore exchange messages with the other busbar of its substation even when it is not energized, but it is still treated as electrically disconnected. Every physical edge value is zero because this structural relation carries information rather than power flow.

The schema-specific encoding is listed in Appendix Table B.3.

Why controlled substations only. The edge represents the actor’s ability to move equipment between busbars. Adding the same edge in a neighboring substation would reveal part of another agent’s action space and would create a nonphysical communication path.

5.2.2 Substation summary nodes (n)

Factor n adds one summary node u_s for each controlled substation. It is connected in both directions to every visible busbar of that substation, which adds one node and $2B$ directed edges. Two busbars can then communicate through the path $i(s, b) \rightarrow u_s \rightarrow i(s, b')$. This serves the same purpose as the direct edge, but takes two message-passing steps instead of one. Only controlled busbars connect to the summary node, so it does not provide an extra path into a neighboring region.

The summary node is initialized from the structure of the substation, not from its identity. The vector $\phi_s \in \mathbb{R}^4$ contains the numbers of busbars, connected lines, generators, and loads. The initialization is

$$u_s^{(0)} = \mathbf{W}_{\text{sub}} \phi_s + \mathbf{b}_{\text{sub}}, \quad \mathbf{W}_{\text{sub}} \in \mathbb{R}^{d_h \times 4}, \mathbf{b}_{\text{sub}} \in \mathbb{R}^{d_h}. \quad (5.1)$$

The node starts with structural information only. Its state becomes dependent on the current observation after it receives messages from the busbars.

The construction is the same in all three graph schemas. A mean/max graph readout counts each u_s as one node. Factors e and n can be enabled together, and both can be combined with the virtual-node readout of Section 5.3.2.

5.3 Graph Readout

After message passing, the actor still needs one fixed-size vector for the full local graph. This step is called graph readout. It can reduce a chosen set of node embeddings with a mean or a maximum, or it can use the state of one learned virtual node. The same output projection and action head are used after either choice.

5.3.1 Mean and maximum graph readout

A graph readout makes two separate choices: which nodes to include and how to combine them. For each readout, let $\mathcal{V}_{\text{sel}}$ denote the selected nodes. Depending on the readout type, this set contains all visible nodes, only controlled nodes, only energized nodes, or nodes that are both controlled and energized. A node is energized only if it touches an active electrical relation. The actor then applies either a mean or a component-wise maximum to the embeddings in $\mathcal{V}_{\text{sel}}$:

$$\bar{\mathbf{h}}_G^{\text{mean}} = \frac{1}{|\mathcal{V}_{\text{sel}}|} \sum_{v \in \mathcal{V}_{\text{sel}}} \mathbf{h}_v^{(K)}, \quad \left(\bar{\mathbf{h}}_G^{\text{max}}\right)_c = \max_{v \in \mathcal{V}_{\text{sel}}} \left(\mathbf{h}_v^{(K)}\right)_c. \quad (5.2)$$

The readout combines all selected node types together; it does not create a separate graph vector for each type. A node outside $\mathcal{V}_{\text{sel}}$ does not contribute to the readout. Combining the four possible node populations with the two reducers gives the eight readouts in Table 5.2.

Table 5.2: The eight final graph readouts, named `[controlled_][energized_]{mean,max}`. Rows choose the readout population, columns the reducer. The virtual-node readout of Section 5.3.2 replaces the selected-node readout entirely.

Selected node population	mean	max
All visible nodes	mean	max
Controlled nodes	controlled_mean	controlled_max
Energized nodes	energized_mean	energized_max
Controlled and energized nodes	controlled_energized_mean	controlled_energized_max

The selected population changes only the final graph readout. It does not remove nodes from message passing. A visible contextual node can still send information to controlled nodes even when it is not part of $\mathcal{V}_{\text{sel}}$. Screen C therefore compares graph readouts, not different information boundaries.

What each axis trades. The node scope and the reducer answer different questions, so their trade-offs should be considered separately (Table 5.3). Before running Screen C, the safest default was `controlled_mean`: the node population is stable, it matches the actor’s region, and it keeps empty busbars that may receive equipment. `controlled_max` is a more risk-sensitive choice when one critical node should dominate the decision.

Table 5.3: Trade-offs along each axis of Table 5.2.

Choice	Main advantage	Main limitation
<i>Reducer</i>		
mean	Smooth regional summary; gradients reach every pooled node.	A rare but critical activation is diluted.
max	Keeps the strongest activation and ignores how many nodes there are.	Sparse gradients, and channel maxima may come from unrelated nodes.
<i>Pooled population</i>		
All visible	Boundary conditions enter the readout directly.	Context can dilute a mean or dominate a max, and the population moves with boundary topology.
Controlled	Aligned with the action domain and fixed for an episode; context still arrives through message passing.	Relies on message passing to carry every useful boundary signal.
Energized	Focuses on the network currently carrying power.	Drops available action targets, and the population changes after each switching action.

5.3.2 Virtual-node graph readout

The virtual-node option replaces the selected-node readout with one learned summary node v_* . It can be used with any of the three graph representations. When substation-summary nodes are enabled, each controlled summary node sends one edge to v_* . Otherwise, every visible controlled busbar sends one edge to it. In all reported runs, these edges point only toward the virtual node. It can collect information but cannot send a message back into the physical graph. Contextual busbars, generators, loads, and line nodes do not connect to it directly; their information must first reach the controlled region through normal message passing.

The initial state $\mathbf{h}_*^{(0)}$ is learned. The same message-passing layers update the virtual node and the physical nodes. After K layers, $\mathbf{h}_*^{(K)}$ replaces the usual graph-readout vector before the output projection, so the embedding size does not change. The added edges are structural. The graph receives one such edge per visible controlled busbar, or one per controlled substation when summary nodes are used.

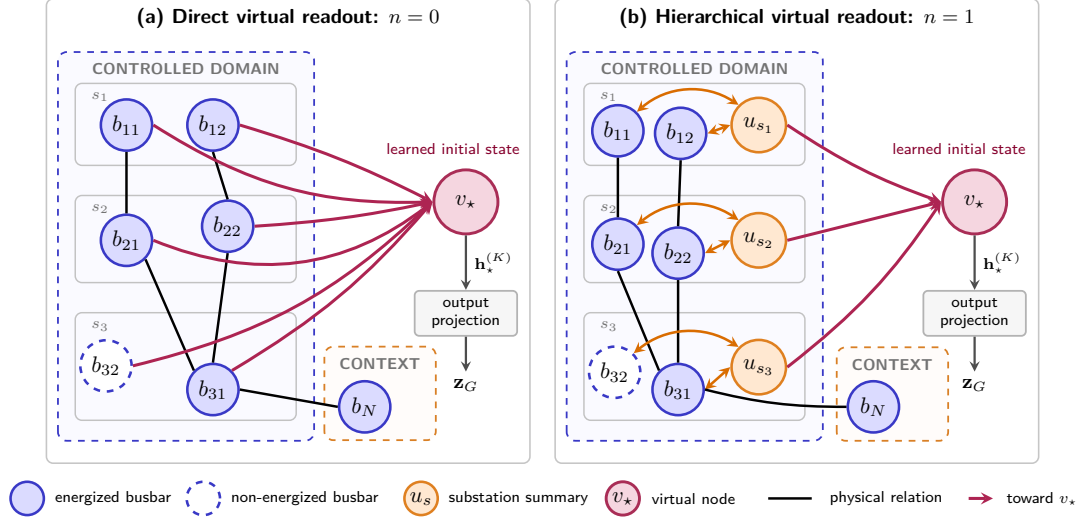


Figure 5.7: Virtual-node readout in the reported one-way setting. Without substation nodes, every controlled busbar, including non-energized b_{32} , sends directly to v_* . With augmentation n , the same busbars communicate through u_s , and only the substation nodes send to v_* . The final state $\mathbf{h}_*^{(K)}$ replaces the selected-node readout.

5.4 Candidate-Action Scoring from the Disaggregated Line-Node Graph

The default action head has one output for every discrete action. Each output is tied to an index, so the network must learn separately what each index means. It does not know which physical objects the action changes. The alternative head in this section scores an action from the graph nodes that it touches. This option is available only for the disaggregated line-node graph of Section 5.1.3. The PPO loss, categorical policy, sampling procedure, and deterministic evaluation remain unchanged.

5.4.1 From action indices to candidate scoring

Let $\mathbf{z}_a$ be the d_z -dimensional graph embedding of agent a . It is obtained from the local node states by the graph readout of Section 5.3, followed by a learned output projection. The default action head receives only this single vector.

With the default head, the logits are produced directly from $\mathbf{z}_a$:

$$\ell_a = \text{MLP}_a(\mathbf{z}_a), \quad \ell_a \in \mathbb{R}^{|\mathcal{A}_a|}. \quad (5.3)$$

The final layer has one row for each action index. The model must therefore learn each action separately, and the head becomes wider when the action space $|\mathcal{A}_a|$ grows.

Candidate scoring replaces these independent output rows with one scalar-valued network shared by all non-idle actions. The idle action can use the same scorer or a separate one:

$$\ell_{a,j} = \text{score} \left(\underbrace{\mathbf{z}_a}_{\text{whole graph}}, \underbrace{\mathbf{c}_j}_{\text{nodes action } j \text{ touches}}, \underbrace{\phi_j}_{\text{optional action descriptor}} \right), \quad j \geq 1. \quad (5.4)$$

The same parameters now score every non-idle action, so the number of learned scorer parameters no longer grows with $|\mathcal{A}_a|$. Two actions are compared through the graph nodes they affect instead of through unrelated output weights.

Table 5.4: Implemented candidate-pooling constructions for the action-specific node context.

Construction	Eligible rows	Aggregation	Width
Joint uniform mean	All nodes touched by the action	One equal-weight mean; node roles are mixed	d_h
Type-separated uniform mean	All nodes touched by the action	One equal-weight mean per type, then concatenation	$4d_h$

5.4.4 Scoring and the idle action

One scalar-output network scores every non-idle action from the global graph embedding, the candidate context, and the optional static descriptor. For a non-idle action, the scorer input and logit are

$$\mathbf{x}_{a,j} = \begin{cases} [\mathbf{z}_a; \mathbf{c}_j; \phi_j], & \text{with the descriptor,} \\ [\mathbf{z}_a; \mathbf{c}_j], & \text{without it,} \end{cases} \quad \ell_{a,j} = \text{MLP}_{\text{score}}(\mathbf{x}_{a,j}), \quad j \geq 1. \quad (5.7)$$

The idle action either reuses this scorer with an empty candidate context or uses a dedicated head, which treats do nothing as a separate global decision:

$$\ell_{a,0} = \begin{cases} \text{MLP}_{\text{score}}(\mathbf{x}_{a,0}) + b_{\emptyset}, & \text{shared, with } \mathbf{c}_0 = \mathbf{0}, \\ \text{MLP}_{\emptyset}(\mathbf{z}_a) + b_{\emptyset}, & \text{dedicated.} \end{cases} \quad (5.8)$$

Here, $b_{\emptyset}$ is a learned scalar offset that calibrates the do-nothing logit relative to the non-idle action logits. It is applied whether action 0 reuses the shared MLP scorer or uses a dedicated MLP head.

5.4.5 Global-maximum and action-delta extension

The Gmax-delta variant keeps the same shared scorer but changes the information given to it. First, the global vector $\mathbf{z}_a$ uses a component-wise maximum over energized nodes instead of a mean. Second, every exact modification in action j is encoded from the object being changed, the mean embedding of its substation busbars, the target busbar, the difference between target and source context, and static operation flags. These tokens are combined into δ_j and appended to the scorer input. The complete branch diagram is shown in Figure 5.9.

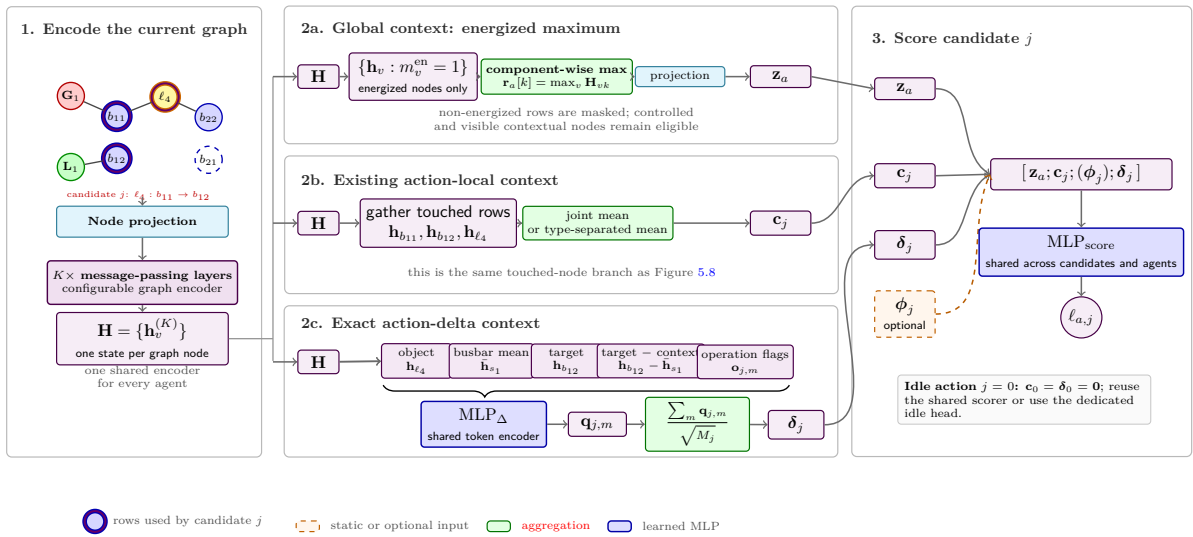


Figure 5.9: Gmax-delta extension of Figure 5.8. The shared graph encoder returns node states $\mathbf{H}$. The upper branch builds the global vector by maximum graph readout over energized nodes; the middle branch keeps the touched-node context $\mathbf{c}_j$; and the lower branch encodes each exact object modification into δ_j .

5.5 Cross-Grid Input Transformations

The graph architecture can be reused on grids of different sizes, but the numerical inputs can still change from one grid to another. The same physical quantity may have a different range, and voltage angles depend on the arbitrary reference bus. I therefore test two optional input transformations: fixed physical scaling and a reference-independent angle representation. They change the values given to the encoder, but not the encoder itself.

5.5.1 Physical scaling

For every continuous feature f , the scaled value is $x'_f = x_f/s_f$, where s_f is a fixed reference in the same unit. This keeps the meaning of the feature and, importantly, keeps zero equal to zero. An empty busbar therefore has zero-valued asset features on every grid. Power is divided by the rating of the largest installed generator, $s_P = \max_g P_g^{\max}$, while angles are divided by 180 degrees. Cooldown, overflow, and maintenance counters are divided by the limits defined by Grid2Op and listed in Table B.4. These scales are fixed when the environment is created; they are not estimated from training data. The loading ratio `rho`, binary indicators, masks, node types, and relation types are already dimensionless and are left unchanged.

5.5.2 Angle representation

An absolute voltage angle needs an arbitrary zero, usually defined by the slack bus. If the same constant is added to every bus angle, the physical state does not change. The difference between the two ends of a line avoids this problem:

$$\Delta\theta_\ell = \theta_\ell^{\text{or}} - \theta_\ell^{\text{ex}}.$$

This line-angle difference does not depend on the reference bus. It is also available for every observed line, whereas an absolute angle is stored only where a generator or load is connected. When this option is enabled, the absolute-angle channels are removed. Chapter 8 shows why this matters: the two grids differ by about 5 to 7° in absolute angle, mostly because they use different slack-bus references. Dividing by a fixed scale cannot remove that offset.

5.6 Action-Space Reduction for Large Grids

The graph representation changes what an agent sees. Action-space reduction changes which decisions it can make. This step is necessary on WCCI, where the full discrete topology-action space is much larger than on bus14. Before MAPPO training, actions are ranked offline using simulated outcomes. States are collected during rollouts whenever the grid is stressed:

$$\rho_{\max} > 0.95.$$

For every collected state s , candidate actions are simulated for one step and compared with doing nothing:

$$\Delta(s, a) = \rho_{\text{after}}(s, a) - \rho_{\text{after}}(s, 0).$$

An action is useful when it reduces the post-action loading by more than ϵ , which means $\Delta(s, a) < -\epsilon$. Its score is the fraction of evaluated states in which it improves on do nothing:

$$\text{score}(a) = \frac{\#\{\text{tested states where } a \text{ improves the grid}\}}{\#\{\text{tested states where } a \text{ is evaluated}\}},$$

The reduced space keeps the k highest-scoring actions for each agent and always keeps action 0. If an agent has fewer than k actions, its complete action set is retained.

The value of k is chosen empirically. I first evaluate each reduced set with a one-step greedy controller. At every risky state, it simulates the available actions and executes the one that improves loading

the most. This is not a learned policy; it checks how much useful control remains after the action space has been cut. Section 7.1 reports the resulting action counts and the values of k used in the transfer experiments.

Chapter Summary

Purpose: Define a graph actor whose representation, graph-level summary, and action interface can be evaluated separately and reused across grids.

- **Graph construction:** The three schemas decide which physical objects receive nodes and what one-hop boundary information is visible to a decentralized actor. The centralized critic still observes the full state.
- **Augmentation and graph readout:** Factors e and n change message passing inside controlled substations. Mean, maximum, selected-node, and virtual readouts determine how the encoded graph becomes one vector.
- **Action interface:** The default head learns one output per action index. Candidate scoring instead shares a scorer across actions and conditions each score on the nodes changed by that action.
- **Transfer preparation:** Fixed physical scales and line-angle differences improve input compatibility, while offline action ranking keeps a manageable WCCI action set whose coverage is checked by a greedy simulator.

Takeaway: Graph representation, information boundary, readout, action scoring, input transformation, and action-space size are separate design choices. This chapter defines the choices needed to interpret the experiments; their exact tensor layouts and implementation inventories are retained in the appendices.

Chapter 6

Graph-Design Screening

This chapter reports six graph-design screens (Screens A–F). All screens are evaluated on seed-0. They ask whether a specific choice in the actor of Chapter 5 changes survival on held-out chronics. Screens A–D evaluate busbar graphs, Screen E evaluates the disaggregated graph, and Screen F evaluates the disaggregated line-node graph with a transferable candidate-action interface. Comparisons are controlled within each screen; results across screens document the progression of the design rather than a single cumulative ablation. **Screens A–E form the source-representation branch later tested through frozen-encoder transfer, whereas Screen F forms the complete-policy transfer branch.**

6.1 Shared Protocol

Table 6.1 separates the common optimization protocol from the boundary and seed settings that differ by campaign. Within a screen, every setting not named as a screened factor is fixed, so each screen is a comparison against its own baseline cell.

Table 6.1: Common optimization settings and campaign-specific boundary and seed settings.

Component	Fixed setting
Environment	bus14 (difficulty 0), topology actions, decentralized agents, normalized observations/rewards, 80/20 train/test chronic split.
Actor	Shared GNN encoder. LayerNorm, node pre-encoder, two 128-unit action-head layers.
Critic	Centralized MLP with three 256-unit hidden layers.
Preprocessing	None.
Budget	15,000,000 environment steps (72 parallel envs, 576 rollout steps).
MAPPO	10 epochs, 16 minibatches. LR 10^{-4} (linear decay). $\gamma = 0.9$, $\lambda = 0.95$, clip 0.2, target KL 0.02, entropy coef 0.01.
Exploration	No initial do-nothing prior; no action-0 logit bonus.
Sparse control	Disabled.
Evaluation	Deterministic, every 82,944 env steps; 20-chronic training-split rollout is used each time, and its mean survival is the checkpoint-selection score.
Seeds	0 (for Screens B, C, D, E, and F).

6.1.1 Endpoint statistics

During training, checkpoints are evaluated through rollouts on the training split. The checkpoint with the highest mean survival in these training-rollout evaluations is retained. This selected checkpoint is then tested on the complete held-out test set of 201 chronics, and that full-test score is reported in the screening results. Every run reaches the 15M-step budget, and all screens report one seed per cell.

6.1.2 Compute

The campaigns were executed on JED (CPU) and IZAR (GPU). Hardware and execution setup were held fixed within each factorial to ensure fair comparisons. Since the screens evaluate exactly one training seed per cell, performance differences are descriptive rather than estimates of between-seed effects.

Training throughput is dominated by stepping the 72 Grid2Op environments rather than the model’s forward and backward passes. For example, in Screen B (Section 6.3), tripling the actor parameter count from 83.6k to 244.9k has a negligible effect on wall-clock time. Parameter count is therefore not the bottleneck when selecting architectures.

6.2 Screen A: Message-Passing Operator

Question

Screen A compared GAT and GINE as the message-passing operator. The purpose was simply to select one operator for the remaining experiments.

Design

Both actors used the busbar graph, two message-passing layers, a hidden width of 128, and the ordinary mean graph readout over visible busbars. The optimization budget and evaluation protocol were otherwise held fixed. Table 6.2 reports the seed-0 full-test score of the best checkpoint over the 201 test chronics.

Table 6.2: Screen A. Operator comparison on the busbar graph with mean graph readout.

Operator	Graph readout	Mean survival (%)
GAT	Mean	91.06
GINE	Mean	94.72

Reading

GINE outperforms GAT by 3.66 percentage points under this matched setting. Given this superior performance, we select GINE as the message-passing operator for all subsequent screens. This empirical advantage is consistent with theoretical expectations, as GINE is known to be highly expressive for distinguishing different graph topologies (aligning with the Weisfeiler-Lehman graph isomorphism test).

Role downstream. GINE is kept as the common encoder for all subsequent screens.

6.3 Screen B: Encoder Depth and Width

Question

This screen measures the effect of message-passing depth K and encoder width d_h . On the busbar graph, depth controls reach: one layer crosses one transmission-line edge, while two layers let each busbar combine information across paths of two physical edges. The graph-level readout then pools all valid node embeddings, so K need not equal the diameter of the local graph. Width controls how much information the node and pooled embeddings can retain.

The result applies directly only to the busbar schema; using the same pair for graphs with explicit asset or line nodes remains an assumption.

Design

A complete 3×4 grid at seed 0, yielding 12 runs. This screen is executed entirely on the busbar graph representation. Table 6.3 lists the screened parameters and their levels. The graph output dimension is matched to the hidden dimension, so a cell varies one width rather than two. Every other setting is inherited from the reference configuration of Section 6.1.

Table 6.3: Screen B. Screened parameters and the values tested. All other settings are held at Table 6.1.

Parameter	Setting	Values tested
Message-passing layers K	<code>gnn_layers</code>	1, 2, 3
Encoder width d_h	<code>gnn_hidden_dim</code>	16, 32, 64, 128

Result

Figure 6.1 reports the complete endpoint grid.

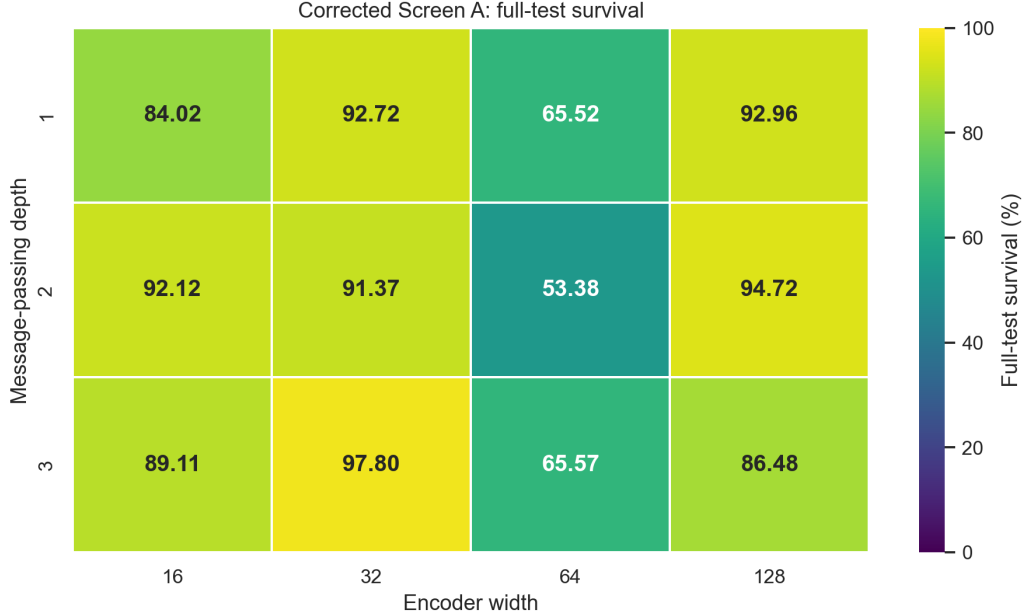


Figure 6.1: Screen B. The same twelve cells as a heatmap of the full-test **mean survival** of each run’s best checkpoint at seed 0.

Reading

No clear architectural rule emerges. While `mp3_h32` achieves the highest single-checkpoint survival (97.80%), neither depth (K) nor width (d_h) show monotonic trends. Depth ranges overlap heavily, and width performance fluctuates unpredictably. Rather than providing a general scaling law, this screen instead serves to identify a single robust operating point for subsequent experiments.

Each endpoint covers all 201 test chronics, but it still reflects a checkpoint selected from roughly 180 ten-episode evaluations and only one training seed. Table 6.4 therefore complements it with metrics computed from the complete training curves.

Table 6.4: Screen B. Convergence measured from the training curves rather than from a single checkpoint. Steps to 90% is the first evaluation at which the smoothed test survival reaches 90%; the curve mean is the time-average of the smoothed curve over training. The final-window mean averages the raw last 20 periodic evaluations, spanning approximately the final 1.58M environment steps. This is the campaign’s late-stability window: it reduces sensitivity to the last few evaluations while remaining local to the converged regime. Dashes mark architectures that never reached the threshold.

Architecture	Steps to 90% (M)	Curve mean (%)	Final-window mean (%)
mp2_h128	6.22	71.40	94.04
mp1_h16	6.88	70.92	84.93
mp3_h32	7.88	69.63	91.28
mp2_h16	8.79	69.16	88.21
mp1_h128	11.03	68.50	91.11
mp1_h32	6.64	63.13	93.77
mp3_h128	–	58.58	80.20
mp1_h64	–	51.60	63.37
mp3_h64	–	49.80	67.58
mp3_h16	–	48.82	60.00
mp2_h32	13.69	46.51	91.08
mp2_h64	–	45.00	53.41

Why carry mp2_h128 forward. Physically, $K = 2$ is a sensible choice because in a small grid like `bus14`, most substations are within two hops of each other. Empirically, `mp2_h128` dominates all convergence and stability metrics (Table 6.4): it crosses 90% survival first, maintains the highest average performance during training, and is significantly more stable in the late-training window. While its absolute best checkpoint is slightly behind `mp3_h32` (94.72% vs 97.80%), this gap is well within the noise variance of a single seed. We therefore select `mp2_h128` as the robust default architecture for the remaining screens.

Role downstream. Two GINE layers and width 128 define the common encoder capacity for the four fixed-head representations transferred in Chapter 9 and for the candidate-action models of Screen F.

6.4 Screen C: Graph Readout Scope and Reducer

Question

The local graph may contain nodes that provide message-passing context without being equally useful in the final graph summary. Should the graph readout include every visible node, only controlled nodes, only energized nodes, or the intersection of controlled and energized nodes? For each scope, should the valid embeddings be averaged or reduced by a per-channel maximum?

The scope mask is applied only at graph readout. Context nodes remain in the message-passing graph, so this screen changes which embeddings form the policy summary without changing what information can propagate to a controlled node.

Design

A 4×2 factorial crosses four readout scopes, namely all visible, controlled, energized, and controlled $\cap$ energized, with mean and max reduction at seed 0. All cells use GINE with $K = 2$ and $d_h = 128$ on the busbar graph. Seven cells are new; the all-visible/mean cell reuses `mp2_h128` from Screen B because it is exactly the same configuration.

Result

Table 6.5: Screen C. Full-test mean survival by readout scope and reducer at seed 0. The all-visible/mean cell is inherited from Screen B.

Readout scope	Reducer	
	Mean	Max
All visible	94.72 [†]	96.31
Controlled	94.60	94.79
Energized	98.93	99.39
Controlled $\cap$ energized	91.45	96.47

[†]Screen-B mp2_h128 reference; not a new run.

Table 6.6: Screen C training-curve diagnostics at seed 0, ranked by curve mean. “To 90%” is the first crossing of 90% by the five-evaluation trailing mean; curve mean is its time-weighted mean over training. Final-20 statistics use the raw final 20 periodic evaluations (approximately 1.58M environment steps).

Scope/reducer	To 90% (M)	Curve mean (%)	Final-20 mean (%)	Final-20 std (pp)
Energized/max	3.40	79.26	97.79	4.12
Energized/mean	8.71	73.59	98.01	2.90
Controlled $\cap$ energized/mean	5.97	72.97	91.18	8.59
All visible/mean [†]	6.22	71.40	94.04	5.29
Controlled/mean	6.47	70.94	91.21	6.08
Controlled $\cap$ energized/max	8.88	68.65	86.31	8.80
All visible/max	4.73	68.38	88.57	8.08
Controlled/max	10.87	68.27	90.67	6.81

Reading

Energized-node readout leads with both reducers. Its mean-readout cell reaches 98.93% and its max-readout cell reaches 99.39%, only 0.46 points apart. Relative to using all visible nodes at graph readout, the energized-only scope improves survival by 4.21 points under mean and 3.08 points under max.

Max is directionally consistent across scopes. For every matched scope, maximum graph readout exceeds mean graph readout on the full test. The gain ranges from 0.19 points (controlled nodes) to 5.01 points (controlled-and-energized intersection). The magnitude depends strongly on scope, but the positive direction does not.

Why carry energized/max forward. It combines the best scope with the reducer that wins every within-scope full-test comparison. Table 6.6 also identifies it as the strongest overall training curve: it is the earliest to cross 90% (3.40M steps) and has the highest time-weighted curve mean (79.26%). While energized/mean has slightly better late-window variance, **energized_max** has the strongest full-test endpoint (99.39%) and is the most consistently supported operating point overall.

Role downstream. Energized/max is retained as the global readout of the fixed-head encoders used in the encoder-transfer branch. We fix it for Screens D and E to prevent readout from becoming an additional confounding factor.

6.5 Screen D: Structural Augmentations of the Busbar Graph

Question

Three optional augmentations add computational structure to the busbar graph without adding any physical measurement: same-substation edges (e), which let the busbars of one substation exchange messages directly; substation-summary nodes (n), which insert one learned hub per substation; and virtual-node readout (v), which replaces the mean graph readout with a learned graph-summary node. Each is motivated by the same intuition, shorten the path between related busbars, so the question is whether any of them earns its cost, and whether they compose.

Design

A complete 2^3 factorial at seed 0, 8 runs. The graph type is busbar throughout and the baseline cell is the unaugmented **e0n0v0**. It has the same $K = 2$, width-128, GINE/**energized_max** design as the reference from Screen C. Cells are named by their three switches, so **e1n0v1** means same-substation edges on, summary nodes off, virtual-node readout on.

Result

The complete factorial result is shown in Figure 6.2.

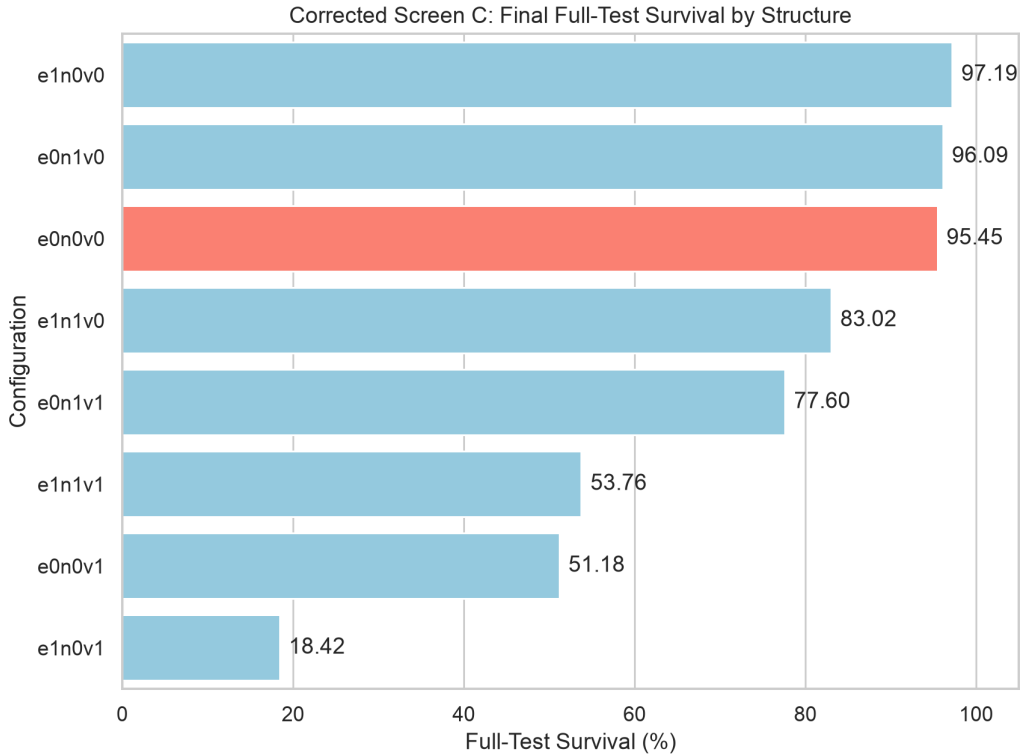


Figure 6.2: Screen D. The factorial results as a bar plot, with the baseline **e0n0v0** highlighted.

Reading

Modest gains do not justify increased complexity. While same-substation edges slightly improve survival (reaching 97.19% vs. the 95.45% baseline¹), this minor benefit degrades erratically when com-

¹This **e0n0v0** baseline matches the best Screen C cell mathematically, but differing hardware between campaigns led to trajectory divergence.

binned with hierarchical factors. Furthermore, virtual-node readouts prove highly destructive (dropping survival to 18–77%), likely because stacking multiple aggregation hubs restricts information flow within our two-layer ($K = 2$) encoder. Consequently, we retain the simple, unaugmented busbar graph for subsequent stages.

Role downstream. Screen D therefore selects the unaugmented **e0n0v0** encoder; the summary-node and virtual-readout variants are discarded. The later frozen-encoder experiment includes **e1n0v0** only as a controlled comparator, not as the selected architecture. This paired test asks whether its small source-grid gain survives a change of grid and therefore whether the added edge structure contains more reusable information.

6.6 Screen E: Generator and Load Message Direction

Question

In the disaggregated graph, each generator and load is a node attached to its busbar. Section 5.1.2 leaves the direction of that attachment open: the asset may send messages to the busbar, receive them from it, or both. The two directions are not symmetric in effect, because the **energized_max** readout pools asset nodes as well as busbars. Does the choice matter, and do the generator and load relations behave independently?

Design

A complete 3×3 factorial over the generator and load directions at seed 0: 9 runs. The graph type is disaggregated throughout; the line and summary directions stay **bidirectional** and are inert for this schema. The encoder matches the reference configuration: GINE, two layers, 128 features, **energized_max** readout, no augmentations, so the only screened factors are the two attachment directions. The baseline cell is bidirectional on both relations. Directions are abbreviated **bi** (bidirectional), **a2b** (asset to busbar), and **b2a** (busbar to asset).

Result

The complete endpoint ranking is shown in Figure 6.3.

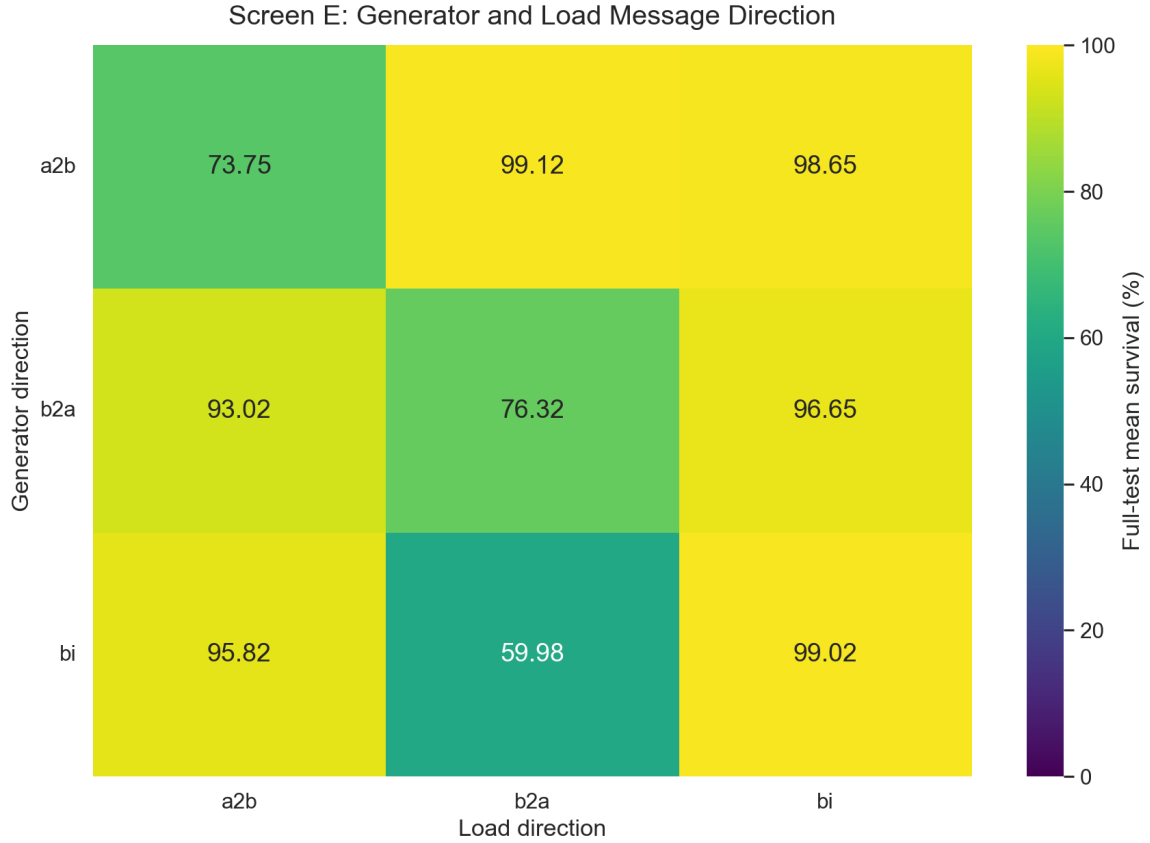


Figure 6.3: Screen E. Full-test mean survival of the best checkpoint by generator direction (rows) and load direction (columns). The baseline cell is **bi/bi**.

Reading

Asymmetric and bidirectional flows are optimal. The best configuration uses **a2b** (asset-to-busbar) for generators and **b2a** (busbar-to-asset) for loads (99.12% survival), closely followed by the fully bidirectional configuration (**bi/bi**) at 99.02% survival. Other symmetric or reversed configurations perform significantly worse. The success of the asymmetric **gen_a2b/load_b2a** flow makes physical sense: because the **energized_max** graph readout includes generator and load nodes, pushing generator information inward to the busbars and pulling busbar information outward to the loads allows the network to correctly route production capacity into the core topology while load nodes contextualize their demand. This prevents the dilution of the pooled graph embedding that occurs when edges incorrectly blur these distinct physical roles.

Role downstream. Both the best configurations are retained for frozen-encoder transfer: **gbi_lbi** and **ga2b_lb2a**. Their 0.10-point source-grid gap is too small at one seed to justify declaring a universal direction, so the WCCI experiment tests whether either relation pattern produces more reusable features.

6.7 Screen F: Candidate-Action Scoring

Question

The candidate-action head of Section 5.4 scores every action from the encoded nodes it modifies rather than from a fixed-width layer over an action index. Three choices in that head are unresolved: how candidate pooling reduces the touched nodes, whether the static action descriptor is appended to the score, and whether the idle action gets its own head. This screen varies all three.

Design

A 2^3 factorial on **bus14**, seed 0, eight runs. Candidate pooling is **mean** or **typed_mean**; the action descriptor is appended (**f1**) or withheld (**f0**); the idle action is scored by a dedicated head (**a0h1**) or by the shared scorer (**a0h0**). Every run uses the disaggregated line-node graph, a two-layer GINE encoder of width 128, and the 15M-step budget of Section 6.1.

Result

Table 6.7 reports the full-test mean survival of each best checkpoint over the 201 held-out chronics.

Table 6.7: Screen F, full-test mean survival of the best checkpoint on 201 held-out chronics. One seed per cell.

Candidate pooling	Descriptor	Idle head	Mean survival (%)	Best checkpoint
mean	f0	a0h1	99.52	14,929,920
typed_mean	f1	a0h0	99.50	14,929,920
mean	f0	a0h0	99.18	14,929,920
mean	f1	a0h1	98.66	14,929,920
mean	f1	a0h0	98.32	14,846,976
typed_mean	f0	a0h1	98.17	14,764,032
typed_mean	f0	a0h0	92.50	12,358,656
typed_mean	f1	a0h1	57.84	11,612,160

Reading

The candidate-action head is a viable actor. Six of the eight cells reach 98.2% or better and three exceed 99%. No single factor strictly separates the performance. The apparent marginal gaps, such as **mean** at 98.9% against **typed_mean** at 87.0%, and **f0** at 97.3% against **f1** at 88.6%, are driven by two low-performing interaction cells; removing the single failure at 57.84% brings the candidate-pooling gap to about a point. At one seed, the defensible statement is that the head works and that the three choices do not cleanly order themselves. Although joint-mean pooling and withholding the descriptor avoid the two weakest cells on **bus14**, this single-seed evidence is not sufficient to select one variant. All eight are therefore retained: choices that are weakly separated on the source grid may differentiate more clearly when transferred to the larger and more difficult WCCI grid.

Role downstream. Screen F supplies the complete-policy branch of Chapter 10. All eight source actors are evaluated after transfer, with each encoder and shared candidate scorer reused together.

How the screens feed the transfer study

Table 6.8: The downstream role of the six screens. A–E contribute to the frozen-encoder experiment; F contributes to complete-policy transfer.

Screen	Evidence retained	Downstream use
A–C	GINE, $K = 2$, width 128, energized/max global readout	Common backbone and readout for the fixed-head encoder-transfer variants
D	e0n0v0 and e1n0v0	Busbar encoders transferred with original and transformed inputs
E	gbi_lbi and ga2b_lb2a	Disaggregated encoders transferred with original and transformed inputs
F	All eight candidate-head variants on the disaggregated line-node graph	Complete-policy zero-shot transfer and target adaptation

Chapter 6 Summary: Two Design Branches

Purpose: To identify effective source-grid graph representations under a controlled fixed action interface, then separately test an action interface designed for complete-policy transfer.

Content: Screens A–E study message-passing operator, encoder capacity, global graph readout, structural augmentation, and asset message direction with a fixed-index action head. Their selected busbar and disaggregated encoders form the frozen-encoder experiment of Chapter 9. Screen F instead tests whether a disaggregated line-node graph with a candidate-action interface can retain strong **bus14** performance while removing the fixed output dimension that prevents complete-policy transfer.

Takeaway & Transfer to WCCI: The screens create two complementary tests. Screens A–E ask whether a learned source representation survives after replacing the incompatible action heads; Screen F asks whether the encoder and action-selection mechanism can cross grids together. The subsequent chapters therefore use:

- **e0n0v0**, **e1n0v0**, **gbi_lbi**, and **ga2b_lb2a** for encoder-only transfer with new WCCI heads;
- the candidate-action scoring (**cas_hl**) runs for complete-policy transfer.

Chapter 7

WCCI36: Target Environment and Controls

The experiments now move from **bus14** to the larger WCCI 2020 grid. This chapter defines the target setting and checks which models can learn on it before testing transfer.

7.1 Target environment

WCCI has more substations, lines and agents than **bus14** (Table 7.1). The agents still control separate areas, but their areas are not equal in size.

Table 7.1: Source and target environments. Both use topology actions, decentralized agents and the same train/test split rule.

	bus14	bus36_wcci_nomaint
Substations	14	36
Lines	20	59
Agents	3	4
Chronics	1,004	2,880
Episode length	up to 8,064 steps	up to 8,062 steps
Maintenance	none	disabled (Section 7.1.1)

7.1.1 Removing maintenance

The WCCI environment normally includes scheduled line maintenance, while **bus14** does not. Maintenance is disabled here so that a transferred policy is not tested on a hazard absent from its source environment. This also keeps the observation spaces aligned.

On the full WCCI test split, removing maintenance raises do-nothing mean survival from 27.3% to 57.3% (Table 7.2). The change is large enough that maintenance-on and maintenance-off results should not be compared directly.

Table 7.2: Do-nothing performance on the full WCCI test split. The 576 chronics are the same in both rows.

Setting	Mean survival	Median survival	Complete episodes
Maintenance enabled	27.3%	12.1%	39/576 (6.8%)
Maintenance disabled	57.3%	100.0%	303/576 (52.6%)

7.1.2 Reduced action spaces

The full decentralized action space contains almost 67,000 actions and is strongly unbalanced across agents. The candidate-action models are therefore evaluated with reduced spaces of $k \in \{32, 64, 128, 256\}$. Here, k is a cap: agents with fewer valid actions keep their complete list.

The greedy survival values below are not computed on the full 576-chronic WCCI test split. They use only the 24 difficult chronics from the fixed 50-chronic evaluation subset defined in Section 7.2.

Table 7.3: Per-agent action counts on `bus36_wcci_nomaint` and one-step greedy mean survival on a subset of the WCCI test split: the 24 difficult chronics from the fixed 50-chronic evaluation set. These are not full-test averages. Agent headings give the number of controlled substations.

Action space	agent_0 (5)	agent_1 (5)	agent_2 (9)	agent_3 (17)	Total	Greedy mean survival
Full list	77	65,642	127	1,119	66,965	–
$k = 32$	32	32	32	32	128	33.1%
$k = 64$	64	64	64	64	256	67.9%
$k = 128$	77	128	127	128	460	68.2%
$k = 256$	77	256	127	256	716	94.9%

A one-step greedy policy is used only as a reference for what each reduced space contains. Its difficult-cohort mean survival rises from 33.1% at $k = 32$ to 94.9% at $k = 256$. A larger action list therefore gives more useful choices, but it also gives the learned policy a harder selection problem. This trade-off is measured in Chapter 10.

7.2 Evaluation protocol

All reported transfer evaluations use the same 50 WCCI chronics. Do nothing completes 26 of them, giving a 52% completion rate. These 26 form the *easy* cohort; the remaining 24 form the *difficult* cohort. Do-nothing mean survival is 56.0% overall and 8.34% on the difficult cohort.

Reporting both values separates two goals: preserving cases that already need no action and improving cases where inaction fails. The number of difficult chronics completed is reported as *rescues*. The one-step greedy policy is a strong reference, not an upper bound, because it queries the simulator at every step.

7.3 Controls trained on WCCI

Figure 7.1 compares three controls trained directly on WCCI: the flat-MLP baseline, the four selected GNN encoders with grid-specific MLP action heads, and the eight GNN candidate-scorer variants. All architecture cells use seed 0. For a family with several cells, the bar reports their arithmetic mean and the whisker spans their minimum and maximum. The number above each bar gives the number of cells included. Figure 7.1 keeps only $k = 64$, because it is roughly the same size the action space on the bus 14 environment.

Incomplete settings excluded: at $k = 32$, only the flat MLP is currently available; at $k = 256$, the flat MLP and selected GNN encoder with an MLP head are available.

Matched WCCI controls trained from scratch at $k = 64$

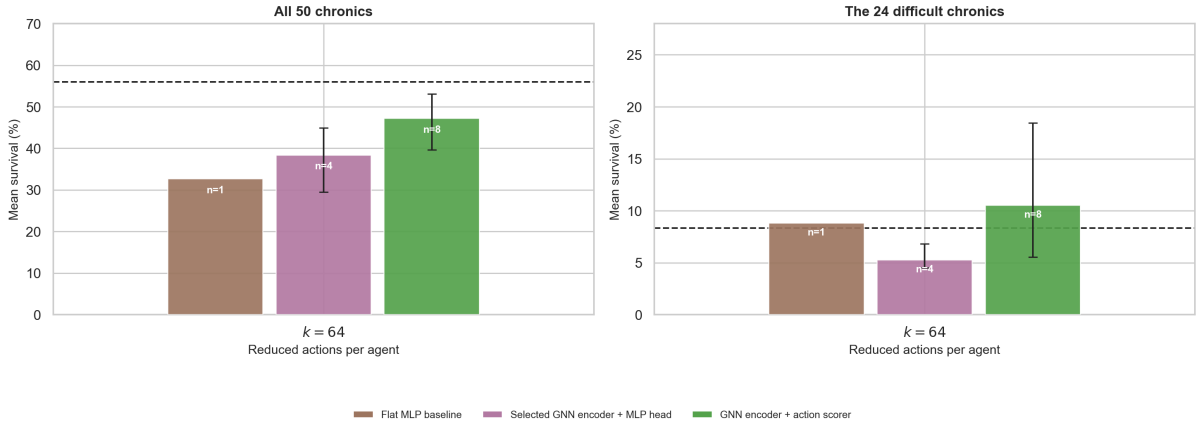


Figure 7.1: Matched comparison of WCCI controls trained from scratch at $k = 64$. Bar colour identifies the model family. Bar height is the family mean, whiskers show the minimum–maximum range when several architecture cells are available, and labels give the number of cells. The dashed line is do nothing.

At the matched $k = 64$ setting, the overall mean across architecture cells is 32.64% for the flat MLP, 38.40% for the four fixed-head GNNs and 47.21% for the eight candidate scorers. All three remain below the 56.0% do-nothing reference. These values are surprisingly low for models retrained directly on the target grid. On the difficult cohort, however, the candidate scorer has the strongest mean at 10.51%, compared with 8.82% for the flat MLP and 5.26% for the fixed-head GNNs. Its best cell reaches 18.46%.

The candidate scorer is also the only graph actor here whose complete policy can cross grids. Its shared scorer is applied to each candidate, so its output size does not depend on the number of agents or actions. A fixed-list head has one output per action index and cannot be reused unchanged on WCCI.

Exact flat-MLP and fixed-list GNN results are given in Appendix A.2.

Chapter 7 Summary: Target Environment and Controls

Purpose: Define the WCCI target and scratch-trained reference controls for the transfer study.

Content: We quantify the maintenance effect, reduce the action space, define a fixed 50-chronic evaluation, and compare the three control families at the matched $k = 64$ setting.

Findings:

- Disabling maintenance raises do-nothing mean survival from 27.3% to 57.3%. On the 24 difficult cases, it reaches only 8.34%.
- Greedy difficult-case survival rises from 33.1% at $k = 32$ to 94.9% at $k = 256$, but larger lists make ranking harder.
- Direct retraining is surprisingly weak: all three $k = 64$ family means remain below do nothing overall. The candidate scorer is strongest on difficult cases, with a 10.51% mean and an 18.46% best cell.

Takeaway: Target-grid retraining alone does not solve WCCI control. These controls provide the reference for encoder reuse and complete-policy transfer.

Chapter 8

Cross-Grid Input Compatibility and Scaling

A GNN can process grids of different sizes, but this alone does not guarantee that its learned representation will transfer. The same input feature must also have a comparable numerical meaning on both grids. For example, an encoder trained on `bus14` may behave poorly on WCCI if a power or angle feature has a different scale or reference. This chapter checks that compatibility before evaluating transfer.

The comparison concerns the graph encoder used by the actors. The critic already receives a normalized flat observation and remains specific to each grid. By contrast, the transferred encoder initially reads raw graph features because physical scaling and running standardization are disabled. Its inputs must therefore be checked directly.

8.1 Measurement protocol

A do-nothing policy is run for 6,000 environment steps on each grid. Each chronic contributes at most 300 steps, which spreads the sample over roughly twenty scenarios instead of concentrating it in one. At every step, the local graph of each agent is rebuilt and all visible nodes are recorded. All the agents are pooled because they use the same encoder. The final sample contains 426,000 node observations for `bus14` and 1,236,000 for WCCI.

Only node features are compared. In this graph schema, the nine edge channels only identify relation types and contain no continuous physical measurements.

Three statistics describe the difference between each pair of feature distributions. Let A denote `bus14` and B denote WCCI, with empirical distributions F_A and F_B , means μ_A and μ_B , and standard deviations σ_A and σ_B . The pooled spread is $\sigma_p = \sqrt{(\sigma_A^2 + \sigma_B^2)/2}$.

$$\text{offset} = \frac{\mu_B - \mu_A}{\sigma_p}, \quad \text{scale ratio} = \frac{\sigma_B}{\sigma_A}, \quad \text{KS} = \sup_x |F_A(x) - F_B(x)|. \quad (8.1)$$

A positive offset means that WCCI has the higher mean. A scale ratio above one means that WCCI has the wider distribution. The Kolmogorov–Smirnov (KS) statistic is the largest difference between the two empirical cumulative distributions and lies between zero and one.

These statistics describe different problems. The offset detects a shift in the centre, while the scale ratio detects a change in spread. KS summarizes the remaining disagreement. A feature is easiest to transfer when its offset is near zero and its scale ratio is near one.

8.2 What the encoder reads

The transfer study uses the disaggregated line-node graph. Its twelve node channels are listed in Table B.2. All node types use the same feature vector, although most channels apply to only one type. The remaining entries are set to zero, so the feature matrix is sparse by construction. For example, `rho` is defined on transmission-line nodes and is zero on busbars, generators and loads. These zeros indicate that the feature does not apply to the node; they are not physical measurements equal to zero.

This sparsity can hide differences between the grids because the shared zeros dominate the complete distribution. The analysis therefore reports each feature both over all rows and over only the rows on which it is defined. For example, the `p` and `theta` results combine generator and load nodes, while `rho` uses transmission-line nodes. Channels that remain constant under the do-nothing policy are kept in the table for completeness, but they provide little information about cross-grid compatibility.

8.3 Results

Table 8.1 reports each statistic in two ways. The first uses all sampled rows, including the structural zeros. The second keeps only rows on which the feature is defined. The second view gives the clearer comparison of the physical measurements.

Table 8.1: Distribution differences between `bus14` and WCCI on the disaggregated line-node graph. Offset is measured in pooled standard deviations, and the scale ratio is WCCI divided by `bus14`. The final three columns keep only rows on which each measurement is defined. Type indicators and `domain_mask` have no such restricted view. A constant channel has no defined scale ratio.

Channel	Zeros (%)		All sampled rows			Applicable rows		
	<code>bus14</code>	WCCI	offset	ratio	KS	offset	ratio	KS
<code>theta</code>	77.5	73.8	+0.653	0.83	0.157	+1.864	1.43	0.678
<code>p</code>	77.8	76.7	−0.037	2.14	0.042	−0.158	2.54	0.201
<code>rho</code>	63.4	63.6	−0.088	0.92	0.053	−0.260	1.05	0.141
<code>line_status</code>	63.4	63.6	−0.005	1.00	0.002	constant at one		
<code>timestep_overflow</code>	100	100	−0.002	0.70	0.000	−0.004	0.70	0.000
<code>time_before_cooldown_line</code>	100	100	N/A	N/A	0.000	constant		
<code>time_before_cooldown_sub</code>	100	100	N/A	N/A	0.000	constant		
<code>is_busbar</code>	60.6	65.0	−0.093	0.98	0.045	N/A		
<code>is_generator</code>	91.5	89.3	+0.076	1.11	0.022	N/A		
<code>is_load</code>	84.5	82.0	+0.066	1.06	0.025	N/A		
<code>is_transmission_line</code>	63.4	63.6	−0.004	1.00	0.002	N/A		
<code>domain_mask</code>	0	0	N/A	N/A	0.000	constant at one		

Over all rows, no channel reaches $KS = 0.2$ because the many zeros hide the differences. On the applicable rows, `theta` has $KS = 0.678$ and an offset of +1.86 standard deviations, while `p` is 2.54 times wider on WCCI. The type indicators differ little, showing a similar mix of busbars, assets and transmission lines.

The raw ranges tell the same story. Power spans $[0, 97]$ MW on `bus14` and $[-281, 399]$ MW on WCCI, where 7.8% of asset rows are negative. These values come mainly from WCCI load profiles: a negative load is a net injection rather than consumption. They are retained because the sign is physical, and they lie outside the range seen during `bus14` training. Angles span $[-11.6, 1.5]$ and $[-7.1, 16.8]$ degrees. Power therefore differs in both scale and support, while the angle distribution mainly shifts its centre.

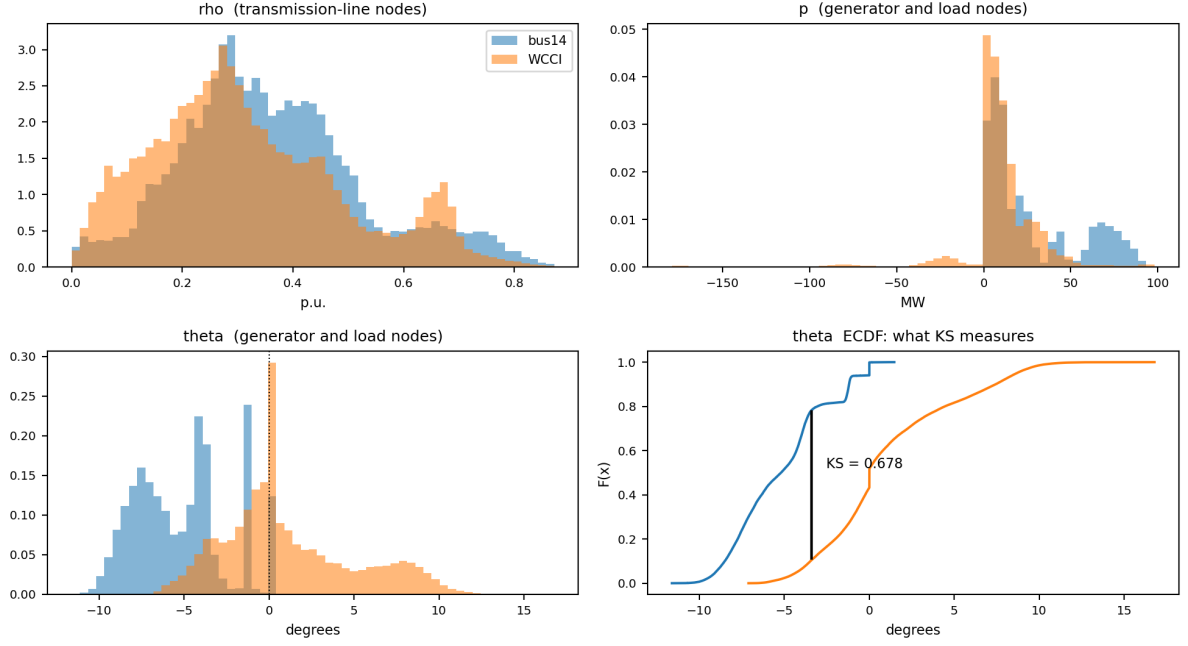


Figure 8.1: Feature distributions on the rows where each channel is defined. ρ is similar on both grids. p has a much wider range on WCCI, while θ is mainly shifted rather than widened. The last panel reports the KS statistic.

8.4 Scale changes and offsets

Figure 8.2 separates a change in centre from a change in spread. Each point represents one feature. The horizontal axis shows the absolute offset, and the vertical axis shows the scale ratio on a logarithmic scale. A feature near the bottom-left corner has a similar centre and spread on both grids. The left panel shows the raw inputs, while the right panel shows the transformed inputs discussed in Section 8.5.4.

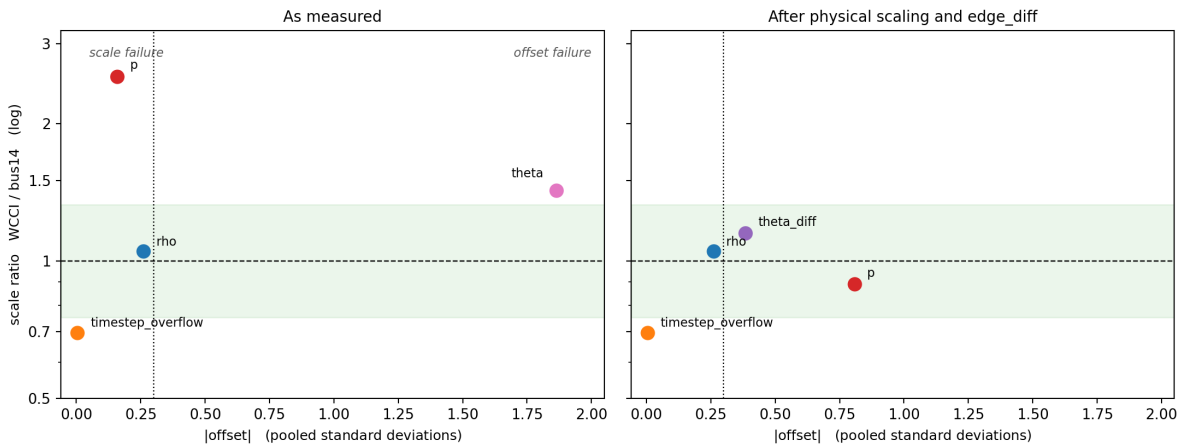


Figure 8.2: Absolute offset against scale ratio on the rows where each channel is defined. The shaded band marks scale ratios close to one, and the dotted vertical line marks an offset of 0.3 pooled standard deviations. The left panel uses raw features. The right panel uses the transformations from Section 8.5. The largest offset falls from 1.86 to 0.81, although p gains a larger offset.

The raw distributions reveal two different mismatches. The `p` channel differs in scale and includes negative WCCI values that were not seen during source training, while `theta` has an offset of 1.86 pooled standard deviations. These observations motivate testing deterministic power scaling and a reference-independent angle representation.

The `rho` channel is already comparable. Its scale ratio is 1.05 and its offset is -0.26 . This is expected because `rho` divides the loading of each transmission line by its own thermal limit. It shows the value of using grid-relative quantities when the physics allows it.

8.5 Transforming the inputs

We explore provides two forms of preprocessing. Physical scaling divides a feature by a fixed constant defined from grid metadata. Running standardization instead estimates a mean and standard deviation during training. The implementation keeps separate statistics for each node type, so the sparse feature blocks do not contaminate one another. It standardizes continuous channels such as `p` and `theta`, but deliberately leaves `rho` unchanged.

8.5.1 Running standardization does not align a new grid

During source training, running standardization can centre and scale the `bus14` inputs and the resulting statistics are saved. In strict zero-shot transfer, they must remain frozen: both grids use the same transformation $(x - \mu_{14})/\sigma_{14}$. This centres `bus14`, but WCCI keeps its relative offset and scale difference. Without clipping, applying the same monotonic affine transformation to both grids also leaves their KS distance unchanged. Running standardization may help source training, but it is not a cross-grid alignment method.

8.5.2 Physical scaling for the power channels

Physical scaling divides the power channels by the largest generator capacity in each environment, as defined in Section 5.5. Scaling leaves the structural zeros at zero, and the divisor is fixed before training. The largest generator is only an approximate measure of grid size, so Table 8.2 compares it with several alternatives. Among the tested choices, it brings the power spreads closest to equality.

Table 8.2: Candidate power normalizers and the resulting `p` scale ratio on asset nodes. A ratio of 1.00 is ideal. Without scaling, the ratio is 2.54.

Divisor	<code>bus14</code>	WCCI	Resulting ratio
Maximum $P_{\max}$ (used)	140	400	0.89
Total $P_{\max}$	540	2,450	0.56
Generator count	6	22	0.69
Median $P_{\max}$	85	71	3.04

8.5.3 Replacing absolute voltage angles

Scaling cannot solve the angle problem. Absolute voltage angles are measured relative to the slack bus, and the two grids use different slack locations. The shift in Figure 8.1 is therefore partly caused by the reference choice rather than by a physical difference between the grids.

The solution is to use the angle difference across each transmission line, as described in Section 5.5.2. This difference does not depend on the slack-bus reference. It is stored on the transmission-line node, which is also more common than the generator and load nodes that carry the absolute angle. Both environments therefore replace absolute angles with line-angle differences. This changes both the physical quantity and the node type that carries it; it is a new representation rather than a rescaling of the original channel.

8.5.4 Results of the transformations

Table 8.3 evaluates both transformations under the same protocol as the raw inputs. The right panel of Figure 8.2 shows the result. The angle difference now behaves similarly to ρ , and the power-scale ratio is within 11 % of one.

Table 8.3: Distribution differences for the original and transformed inputs after enabling `gnn_angle_representation = edge_diff` and `gnn_physical_scaling = true`. Statistics are computed on the node rows where each channel is defined. The offset is retained because scaling each grid by a different constant can also move the centre.

Channel	Original			Transformed		
	offset	ratio	KS	offset	ratio	KS
theta	+1.864	1.43	0.678	replaced		
theta_diff		N/A		−0.383	1.15	0.230
p	−0.158	2.54	0.201	−0.808	0.89	0.511
rho	−0.260	1.05	0.141	−0.260	1.05	0.141

The angle transformation clearly improves cross-grid compatibility. Replacing the absolute angle with a reference-independent difference reduces KS from 0.678 to 0.230, while the scale ratio remains close to one. Because the two channels describe different quantities on different node types, this is a comparison between representations rather than a direct normalization of the same variable.

The power transformation involves a trade-off. Scaling improves the spread ratio from 2.54 to 0.89, but it increases the offset from -0.158 to -0.808 and raises KS from 0.201 to 0.511. The reason is that the mean sampled generator output represents about 39 % of the largest generator capacity on **bus14**, but only 20 % on WCCI. These values do not describe the utilization of each individual generator. Dividing by the largest generator capacity therefore places typical outputs at different numerical levels on the two grids.

The evidence supports the angle representation, while power scaling remains uncertain. The transformed-input condition combines both transformations as an experimental condition, while the raw-input condition keeps the original features as a control.

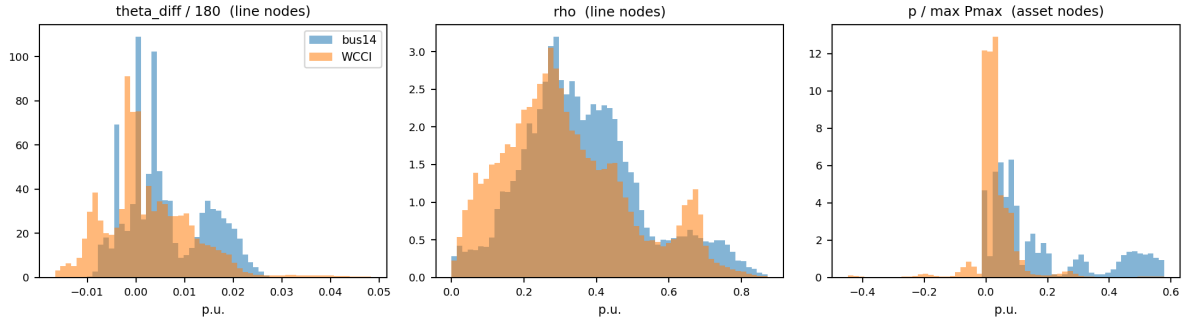


Figure 8.3: Transformed feature distributions. The angle difference is centred in a similar region on both grids, and the scaled power channels now have similar widths. Compare with the raw distributions in Figure 8.1.

8.6 Inputs retained for transfer

The transfer experiments retain two input conditions. The raw-input condition uses the original graph features. The transformed-input condition divides power by the largest generator on each grid and replaces absolute voltage angles with line-angle differences. Running standardization is disabled because frozen source statistics cannot align WCCI in strict zero-shot transfer. Comparing otherwise matched raw-input and transformed-input models isolates the effect of deterministic input preprocessing. The

later bridge experiments separately study the effect of source pretraining in Chapters 9 and 10.

Chapter 8 Summary: Cross-Grid Input Compatibility and Scaling

Purpose: Check whether the shared encoder receives comparable numerical inputs on **bus14** and WCCI.

Content: Do-nothing rollouts provide 426,000 **bus14** and 1,236,000 WCCI graph-node entries. Each feature is compared using its offset, scale ratio and KS statistic.

Findings:

- **rho** is already comparable because each line loading is divided by its own thermal limit.
- WCCI power has a wider range and includes negative net injections that are absent from the sampled **bus14** inputs.
- Replacing absolute angles with line-angle differences reduces angle KS from 0.678 to 0.230.
- Power scaling improves the spread ratio from 2.54 to 0.89, but increases the offset from -0.158 to -0.808 .
- Type-aware running standardization preserves sparse zeros, but frozen source statistics cannot align WCCI in strict zero-shot transfer.

Takeaway: The angle transformation is supported by the distribution analysis, whereas power scaling has mixed effects. The raw-input and transformed-input conditions are therefore both evaluated. Target-specific standardization would be a separate calibration experiment rather than strict zero-shot transfer.

Chapter 9

Encoder-Only Transfer Across Grids

Screens A–E give graph encoders that work well on `bus14`. Their action heads cannot be reused on WCCI because each output corresponds to one action from the `bus14` action list. In this chapter, I keep the encoder, freeze it, remove the old action heads, and learn new heads on WCCI. Chapter 10 studies the other case where the encoder and the candidate-action scorer are transferred together.

9.1 Question and hypotheses

The main question is simple: does an encoder trained on `bus14` give better WCCI features than an encoder that has not been trained on the source grid?

The experiment checks three things:

- source pretraining against random frozen features;
- original inputs against transformed inputs;
- a frozen encoder against the same architecture trained end to end on WCCI.

Four representations are kept from Screens D and E:

- `e0n0v0`: the basic busbar graph;
- `e1n0v0`: the busbar graph with same-substation edges;
- `gbi_1bi`: the disaggregated graph with bidirectional generator and load edges;
- `ga2b_1b2a`: the disaggregated graph with asymmetric generator and load edges.

They all use the common choices from Screens A–C: GINE, two message-passing layers, width 128, and energized/max graph readout. I keep several representations because their results are very close on `bus14`, but they may behave differently on WCCI.

9.2 Transferred components and controls

The setup is the following:

- **Frozen:** node and edge projections, type embeddings when they exist, the GINE layers, normalization layers, and the final representation projection.
- **Replaced:** the old `bus14` action heads.
- **Trained on WCCI:** one new action head per agent and a new centralized critic.

The four training routes are shown in Table 9.1.

Table 9.1: Training routes for the encoder-only transfer study.

Route	Encoder initialization	Trainable actor parameters on WCCI	Purpose
Target scratch	Random	Encoder and new heads	Target-learning reference
Random frozen	Random	New heads only	Frozen-feature control
Original-input frozen transfer	Existing original-input bus14 checkpoint	New heads only	Original-input encoder reuse
Transformed-input frozen transfer	Retrained transformed-input bus14 checkpoint	New heads only	Transformed-input encoder reuse

The random-frozen route is important. A new WCCI head may learn something even if its encoder is random. A source encoder only shows useful transfer if it does better than this control. The scratch route shows what the architecture can learn when all its weights are allowed to change on WCCI.

Table 9.2: Protocol fields that must be fixed before the matched WCCI experiment. Every route must use the same target values.

Field	Fixed value
WCCI reduced action space	[Pending: action cap and dataset path]
WCCI training budget	[Pending: environment steps]
Source and target seeds	[Pending: seed set]
Head optimizer and learning rate	[Pending: matched head-training recipe]
Checkpoint selection	[Pending: selection score and rollout frequency]
Final evaluation	50 fixed test chronics; 24 difficult chronics
Maintenance and heuristic	Maintenance disabled; [Pending: gate condition]

For the original-input route, I use the existing checkpoints from Screens A–E. For the transformed-input route, the same architectures are retrained on **bus14** with physical power scaling and line-angle differences. The same transformation is then used on WCCI. It cannot be added only during transfer because the frozen encoder must receive the same kind of input that it saw during source training.

9.3 Results

9.3.1 Learning dynamics

The learning curves will compare:

- target scratch;
- random frozen;
- original-input frozen transfer;
- transformed-input frozen transfer.

All routes need the same WCCI budget and the same evaluation steps. The main thing to check is whether a source encoder learns faster or reaches a better score than the random-frozen control.

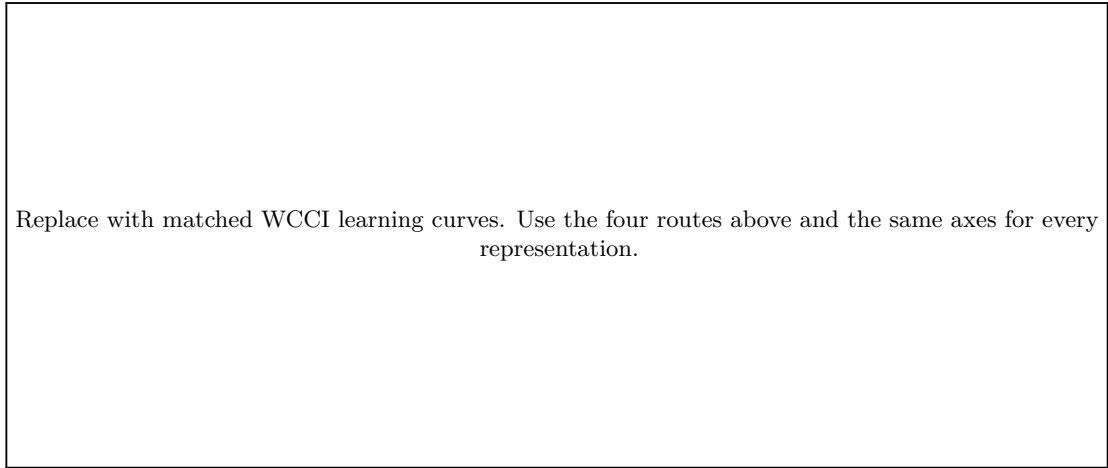


Figure 9.1: Learning curves for the encoder-transfer routes.

9.3.2 Full-test comparison

The final comparison reports:

- mean survival on all 50 chronics;
- mean survival on the 24 difficult chronics;
- difficult chronics completed by the model;
- easy chronics that remain completed.

Table 9.3: Mean survival on the 24 difficult WCCI chronics. Values will be added after the matched evaluations are complete.

Representation	Target scratch	Random frozen	Original-input frozen	Transformed-input frozen
e0n0v0	TBD	TBD	TBD	TBD
e1n0v0	TBD	TBD	TBD	TBD
gbi_lbi	TBD	TBD	TBD	TBD
ga2b_lb2a	TBD	TBD	TBD	TBD

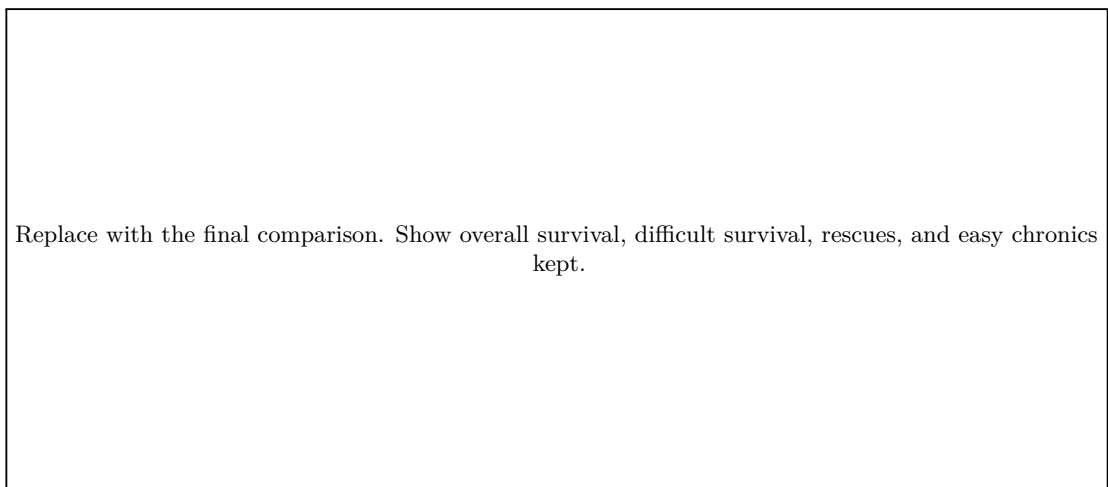


Figure 9.2: Final WCCI comparison of the four representations and four training routes. Do nothing and scratch remain visible as references.

Result to add:

- Does a pretrained frozen encoder beat the random-frozen control?
- Are transformed inputs better than original inputs?
- How far is the best frozen encoder from target scratch?

Chapter 9 Summary: Encoder-Only Transfer

- **Purpose:** Check if an encoder learned on `bus14` still helps after the action heads are replaced on `WCCI`.
- **Compared:** Four source encoders, original and transformed inputs, a random-frozen control, and training from scratch.
- **Finding:** [Pending: main difficult-chronic result and learning-speed result]
- **Takeaway:** [Pending: whether the encoder transfers and whether transformed inputs help]

Chapter 10

Complete-Policy Transfer and Target Adaptation

Chapter 9 tested whether a source encoder remains useful after its incompatible action heads are replaced. This chapter asks the stricter question: can the shared candidate-action actor learned on **bus14** control WCCI while retaining both its encoder and its action-selection interface? It first evaluates the complete actor without target training, then studies target adaptation.

10.1 Comparison rules

The transferred object is the complete candidate-action actor, not only its graph encoder. The actor varies along four binary choices: original or transformed inputs, mean or typed-mean candidate pooling, action descriptor off or on, and a shared or dedicated idle-action head. Table 10.1 defines the three training routes.

Table 10.1: Training routes used in the transfer study.

Route	Initial actor	WCCI training
Zero-shot	Shared bus14 actor	None
From scratch	Random weights	15M target steps
Fine-tuned	Shared bus14 actor	5M target steps

Final checkpoints are used for actors trained on WCCI. Zero-shot evaluation uses the source checkpoint selected on **bus14**. The optional heuristic blocks an agent’s action when its local maximum line loading is below 0.95. Every comparison uses the same 50 chronics and seed 0.

10.2 What controls zero-shot transfer?

Figure 10.1 separates three questions. The top panel gives mean survival over all 50 chronics. The lower-left panel restricts the score to the 24 difficult chronics. The lower-right panel then adjusts that difficult-cohort score to the greedy reference available at the same k :

$$R_{k,\pi} = 100 \frac{S_{\pi} - S_{\text{DN}}}{S_{\text{greedy},k} - S_{\text{DN}}}. \quad (10.1)$$

Here 0% means do-nothing performance and 100% means the greedy result at the same action count. A negative value is worse than do nothing. This allows to judge the policy according the maximum performance empirically found with this action-space size.

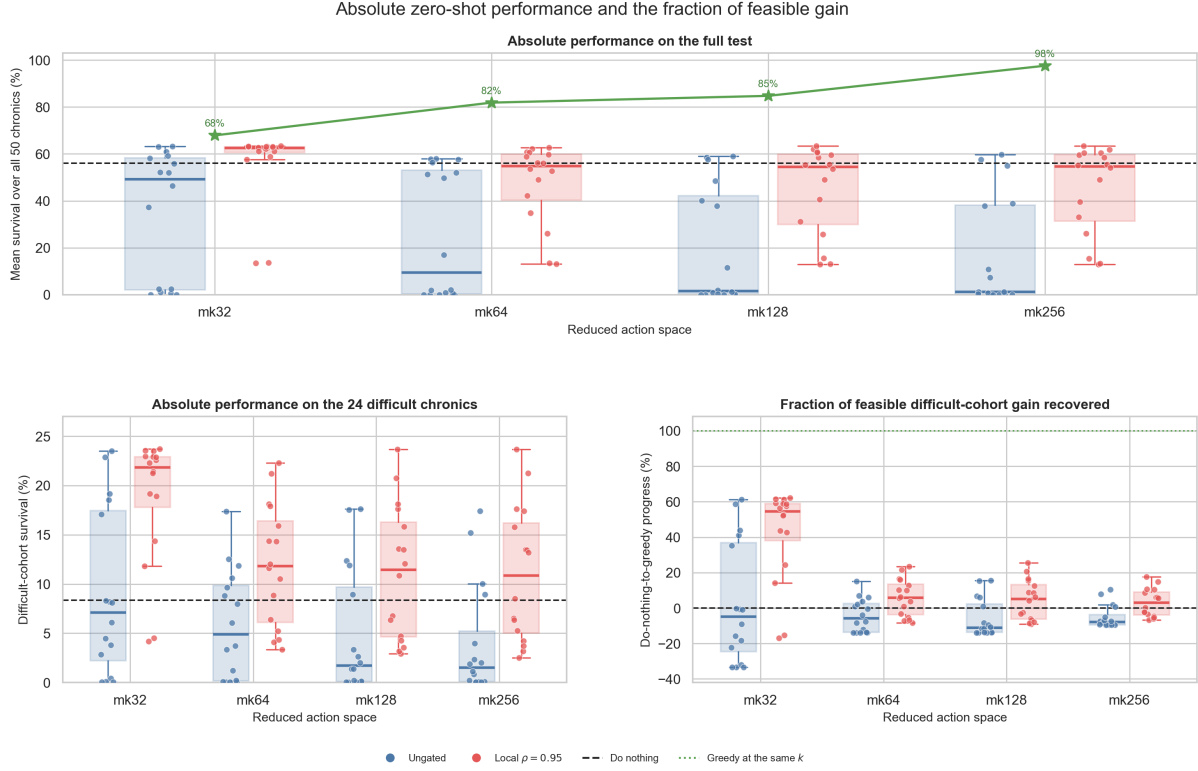


Figure 10.1: Absolute and feasible-relative zero-shot performance. Each box contains all 16 architectures across the raw-input and transformed-input conditions. Green stars give the greedy full-test score and dashed lines give do nothing. The lower-right score measures progress from do nothing to the greedy reference at the same action-space size.

The distributions show that absolute and feasible-relative performance answer different questions. With the heuristic, the median of the models’ full-test mean-survival scores is 62.5% at $k = 32$ and about 54.5% for the larger action spaces; the median difficult-cohort mean survival across models falls from 21.9% to roughly 11%. Relative to the same- k greedy reference, the median recovers 54.6% of the available gain at $k = 32$, but only 5.8%, 5.2% and 2.9% at $k = 64, 128, 256$. The gate helps, but the actors exploit a smaller fraction of what becomes feasible as the action set grows. Ungated median performance remains below do nothing at every k .

Figure 10.2 then separates the architecture effects by gate condition. Typed-mean candidate pooling is the clearest positive effect: +5.4 points ungated and +8.7 points gated. Physical scaling adds about one point in both cases. The descriptor is small on average, while the dedicated idle head is negative ungated and nearly neutral gated.

Architecture effects depend on the gate condition

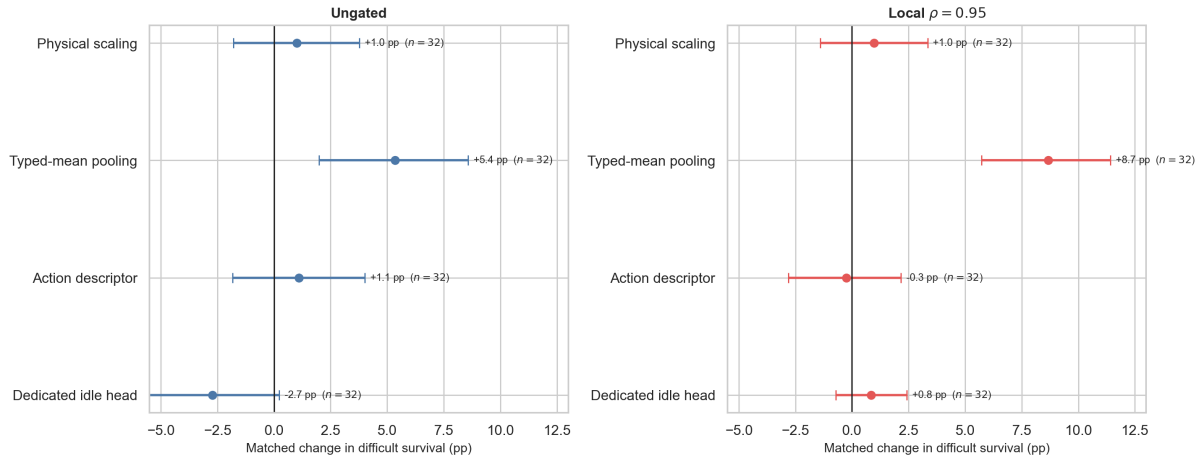


Figure 10.2: Matched architecture effects on the difficult cohort, shown separately for ungated actors and the local- $\rho = 0.95$ heuristic. Positive values improve survival. Error bars are bootstrap intervals over matched architecture cells; n is the number of available pairs.

These comparisons use one seed. They identify useful directions but do not establish a final architecture ranking.

10.3 Policy changes and the local heuristic

Two changes are tested around the zero-shot actor. The first uses the global-maximum readout and exact action-delta encoder of Figure 5.9. The second uses an adaptive intervention budget. Figure 10.3 first shows their absolute performance in one view. Each action-space cluster contains four distributions: the two policy changes, with and without the local- ρ heuristic.

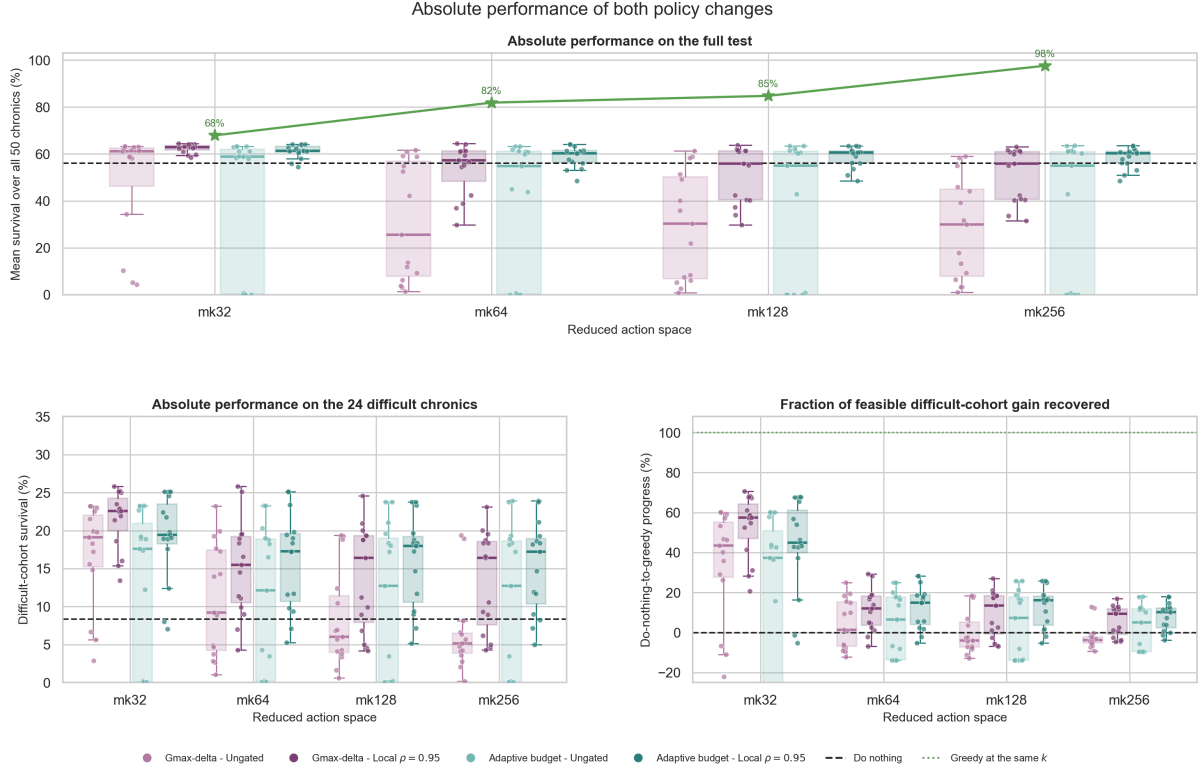


Figure 10.3: Absolute performance of the Gmax-delta and adaptive-budget actors at each action-space size, with four distributions per cluster. Boxes and points summarize the architecture cells for ungated and local- $\rho = 0.95$ evaluation. The dashed line marks do-nothing; the green reference gives the greedy ceiling or, in the relative panel, its full attainable gain.

The absolute view shows whether a positive change is large enough to produce a useful policy. On difficult chronics, ungated Gmax-delta falls from a 19.1% median at $k = 32$ to 5.1% at $k = 256$, whereas the adaptive budget retains 12.7% at $k = 256$. With the heuristic, both modified actors remain above the do-nothing reference at every k . Figure 10.4 then isolates the matched change from the plain actor and the effect of the local- ρ heuristic.

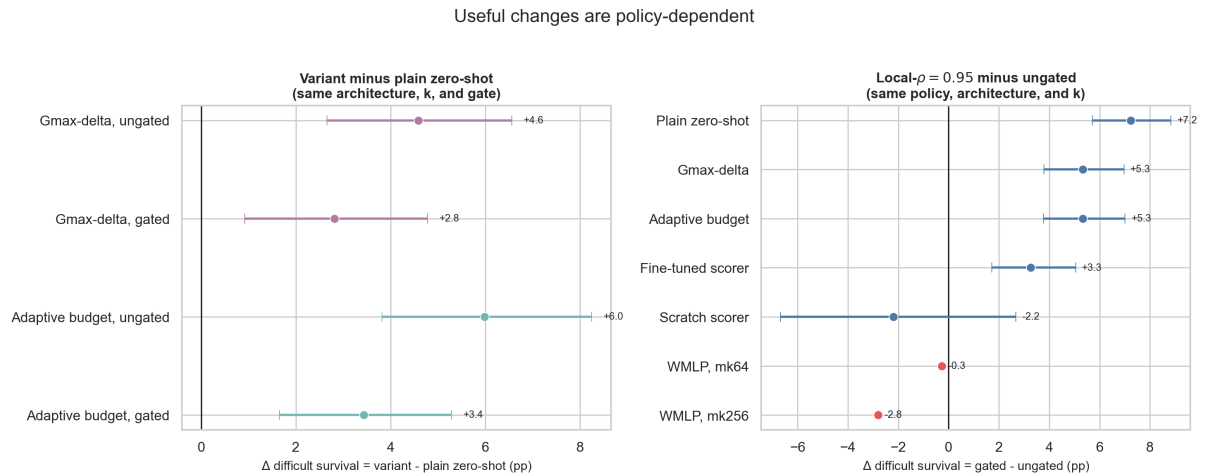


Figure 10.4: Paired changes on the difficult cohort. In the left panel, each policy variant is compared with the plain zero-shot actor using the same architecture, action-space size and gate condition. In the right panel, local- $\rho = 0.95$ evaluation is compared with ungated evaluation for the same policy, architecture and action-space size. Points are mean paired differences and error bars are 95% bootstrap intervals. Positive values favour the first condition named in each panel; zero is the reference.

Without the heuristic, Gmax-delta adds 4.6 points and the adaptive budget adds 6.0 points. Across gate conditions, the heuristic adds 7.2 points to the plain zero-shot actor and 5.3 points to both modified actors. It is not a universal safety layer: it reduces the scratch scorer by 2.2 points and the flat MLP by up to 2.8 points. The gate must therefore be checked with each policy.

The plain block contains all 64 matched architecture- k pairs. Gmax-delta and the adaptive budget each contain 60 pairs because their complete designs use 15 architectures rather than 16.

10.4 Does candidate-policy target training help?

Figure 10.5 compares the same eight transformed-input architectures at $k = 64$. The left panel tracks each architecture across zero-shot, scratch and fine-tuned routes. The right panel shows the main trade-off: mean survival on the difficult cohort against the number of easy chronics kept.

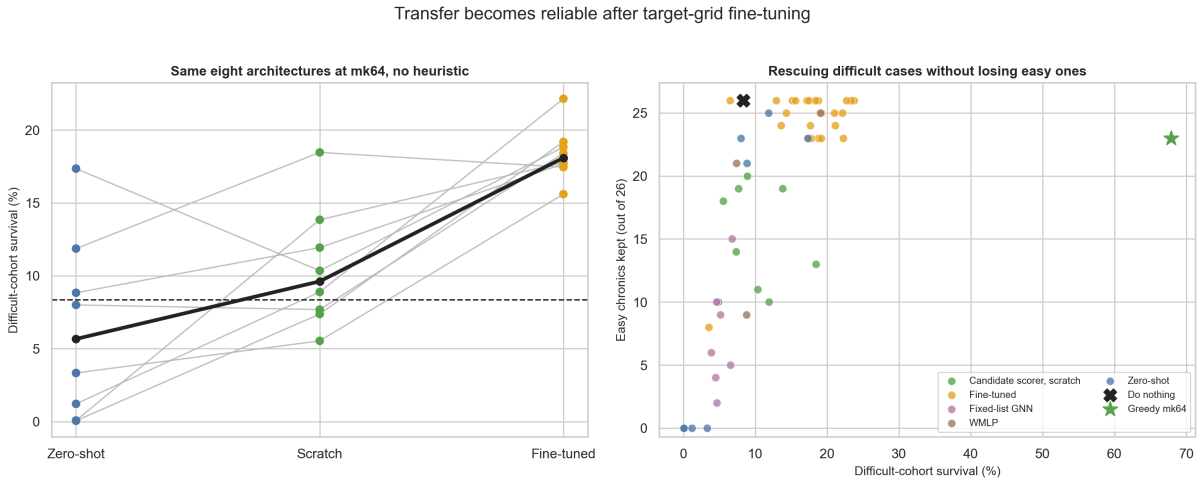


Figure 10.5: Effect of target-grid training. Grey lines join the same architecture in the left panel. The black cross and green star in the right panel are do nothing and the greedy $k = 64$ reference.

Fine-tuning is the most consistent route. It beats training from scratch on seven of the eight difficult-cohort comparisons, with a mean gain of 7.9 points. It also beats zero-shot transfer in all eight cells, by 12.0 points on average. The median difficult-cohort mean survival across architectures rises from 5.7% zero-shot to 9.6% from scratch and 18.1% after fine-tuning.

The right panel adds an important qualification. Some actors improve difficult chronics while losing cases that do nothing already solves. The useful region is therefore the upper-right corner, not simply the highest difficult-cohort score. The target-trained controls remain far below the greedy policy, showing that much more performance is available in the action space.

10.5 Limits of the study

The study uses one training seed and 50 evaluation chronics, which is not sufficient for generalization. Bootstrap intervals describe variation across architecture cells, not across independent training runs. The results support comparisons inside the matched blocks, but small gaps between unrelated blocks should not be overinterpreted. Full matrices and exact control tables are provided in Appendix A.2.

Chapter 10 Summary: Complete-Policy Transfer and Adaptation

Purpose: To determine how well the complete shared candidate-action actor learned on `bus14` transfers to WCCI, identify which architectural and decision-time choices improve zero-shot behavior, and measure whether WCCI training benefits from fine-tuning.

Content: Unlike Chapter 9, the source action interface is not replaced. We evaluate 16 architecture cells across the raw-input and transformed-input conditions over $k \in \{32, 64, 128, 256\}$, with and without a local- $\rho = 0.95$ gate, using both absolute mean survival and progress from do nothing to the same- k greedy reference. We then test Gmax-delta and adaptive-budget policy changes before comparing the same eight transformed-input architectures at $k = 64$ under zero-shot transfer, training from scratch and target fine-tuning.

Findings:

- Zero-shot transfer is strongest with the smallest action set. With the gate, $k = 32$ reaches 62.5% overall and 21.9% on difficult chronics, recovering 54.6% of the available same- k gain; the recovered fraction falls to 2.9% at $k = 256$. Ungated medians remain below do nothing.
- Typed-mean candidate pooling is the clearest positive architecture effect, adding 5.4 difficult-cohort mean-survival points ungated and 8.7 gated. Physical scaling adds about one point, while the descriptor is small on average and a dedicated idle head is harmful ungated and nearly neutral gated.
- Gmax-delta and the adaptive budget add 4.6 and 6.0 matched points without the gate. The gate improves 58 of 64 plain cells and adds 7.2 points on average, but it hurts the scratch scorer and flat MLP; it is therefore policy-dependent rather than a universal safety mechanism.
- Fine-tuning is the strongest adaptation route. It beats zero-shot in all eight matched cells by 12.0 points on average and beats scratch in seven cells by 7.9 points. Median difficult-cohort mean survival across architectures progresses from 5.7% zero-shot to 9.6% scratch and 18.1% fine-tuned.

Takeaway: Reliable transfer requires controlling the action-selection problem, not merely enlarging the available action set. Small candidate sets and typed-mean candidate pooling are the most robust zero-shot choices; local gating must be validated per policy; and conservative target fine-tuning is more effective than either zero-shot deployment or training the same actor from scratch. These conclusions are based on one seed and 50 chronics, so matched effects are more credible than small gaps between unrelated cells.

Chapter 11

Discussion and Conclusions

This thesis asks whether graph-based decentralized policies can control one grid and remain useful on a larger one. The work addresses this question in four stages: stable learning, sparse intervention, graph design, and cross-grid transfer.

11.1 Research questions revisited

1. Which changes stabilize decentralized MAPPO? The tuned flat actor reaches high survival earlier and more consistently than the initial implementation. Its final mean survival is about 93% on `bus14`. Reward normalization, the critic update, and the optimization settings have a clearer effect than simply increasing the model size. An initial bias toward do nothing is not consistently helpful: a strong bias delays learning, while a weaker bias remains unstable. The zero-bias model is therefore retained as a stable reference, even though it does not have the highest endpoint in every comparison.

2. Can unnecessary interventions be reduced? Yes, at least on the source grid. The local-safe adaptive intervention budget reaches about 98.4% mean survival while selecting action 0 in about 98% of decisions. The baseline reaches 93.67% survival and selects action 0 in only 43.4% of decisions. The agents can therefore learn to intervene less often without losing performance. A fixed penalty also gives good results, whereas the learned gate does not improve both survival and intervention frequency at the same time.

3. Which graph design provides a useful local actor? Several graph actors exceed 99% mean survival on `bus14`. For the fixed-head encoder branch, the source-grid screens retain GINE message passing, two layers of width 128, and an energized-node maximum graph readout. Several busbar and disaggregated representations are then kept because their small source-grid differences may become more important after transfer. The candidate-action head also works well: six of its eight cells exceed 98.2% and three exceed 99%. More importantly, its shared scorer can rank action lists of different sizes, which makes complete-policy transfer possible.

4. Do graph-policy weights transfer to WCCI? The complete shared encoder and candidate scorer can run on WCCI without any change to their parameter dimensions. This confirms structural transfer, but the transferred behavior remains limited. On the 24 difficult chronics, do nothing reaches 8.34% mean survival. With the local- $\rho = 0.95$ heuristic, the median zero-shot actor reaches 21.9% at $k = 32$, while the best evaluated actor reaches 25.8%. Performance decreases as the candidate list grows even though the greedy reference improves. The main difficulty is therefore not the lack of useful actions, but ranking them correctly. In the matched $k = 64$ comparison, fine-tuning raises median difficult-chronic survival to 18.1%, compared with 9.6% from scratch and 5.7% zero-shot. Source initialization is useful, but target adaptation is still needed.

This answer will be completed after Chapter 9 establishes whether a frozen source encoder outperforms the random-frozen control when the WCCI action heads are releared.

11.2 What the experiments show together

High source-grid survival does not guarantee successful transfer. The screens in Chapter 6 show that the graph encoders and candidate scorer can learn the `bus14` task. The WCCI experiments then test two different forms of reuse. Encoder-only transfer asks whether the learned representation remains useful after replacing the grid-specific head. Complete-policy transfer also reuses the action-selection mechanism.

Together, the experiments give three practical lessons. First, compatible parameter dimensions do not guarantee compatible behavior. The input convention, candidate actions, and intervention frequency all matter. Second, a smaller candidate set can help a learned policy even when a larger set contains better greedy actions, because the policy must still rank those actions correctly. Third, the local heuristic is policy-dependent: it improves most zero-shot actors but can hurt policies trained directly on WCCI. Overall survival should therefore be read together with difficult-chronic survival, since many of the easy chronics are already completed by doing nothing.

11.3 Limitations and next steps

The main architecture screens use one training seed, and the controlled WCCI comparison uses 50 chronics from a single source-target grid pair. Maintenance is disabled on WCCI and the policies use reduced action spaces. Scratch and fine-tuned actors also receive different training budgets. Their comparison therefore supports the observed performance ordering, but not a strict claim about sample efficiency. Finally, the one-step greedy controller is a useful reference, not an upper bound on long-term control.

The next experiments should focus on the finalists rather than repeat the full architecture search. They should use additional seeds, a larger untouched WCCI cohort, and equal budgets for scratch training and fine-tuning. Testing with maintenance and on a second target grid would show whether the conclusions are more general. Better candidate ranking is also a priority, because the largest remaining gap lies between the learned policies and the useful actions already available to them.

11.4 Conclusion

Graph and candidate-action architectures remove the dimensional barrier that prevents a fixed-width decentralized actor from being reused across grids. However, structural compatibility alone does not guarantee reliable control on a new network. In the `bus14`-to-WCCI study, useful transfer also depends on compatible inputs, a manageable action set, candidate-aware scoring, policy-specific intervention control, and target adaptation. The main conclusion is therefore not that one graph architecture solves transfer. It is that a transferable policy can retain meaningful zero-shot performance and provide a better starting point for adaptation than learning the same control problem again from random weights.

Appendix A

Additional Experimental Evidence

A.1 Sparse Intervention Control

The joint survival/action-0 comparison is retained in Figure 4.1; the material below provides the compact experimental settings and exact condition means.

A.1.1 Compact experimental details

The two deterministic gate decoders are defined in Section 4.2. The gate-plus-budget diagnostic also separates the gate and non-idle entropy terms:

$$H_i = \alpha_{gate}H(g_i) + \alpha_{nonidle}H(b_i),$$

instead of using the coupled form $H_i = H(g_i) + P(g_i = 1)H(b_i)$. This avoids rewarding intervention merely because a larger intervention probability exposes more non-idle entropy.

The fixed-penalty experiment uses flat and gated actors, $\lambda_{fixed} \in \{0, 0.001, 0.003, 0.01\}$, and seeds $\{0, 1, 2\}$, giving 24 runs. The tested conditions are flat local-safe budgets $d \in \{0.10, 0.20, 0.35\}$, a gated local-safe budget with $d = 0.20$, and a flat plain non-idle budget with $d = 0.20$.

For the teacher-student diagnostic, the teacher is the MAPPO policy evaluated with the local rho heuristic. The student is trained to reproduce the teacher’s executed actions and is evaluated without the heuristic. The two reported students use balanced non-idle fractions of 0.50 and 0.20, with non-idle loss weights of 5 and 3, and auxiliary weights of 0.50 and 0.25, respectively.

A.1.2 Exact aggregate results

Table A.1 reports the condition means over three seeds. The action-0 value is the arithmetic mean of the three agents’ fractions. Figure 4.1 shows the same two metrics together with the individual seeds.

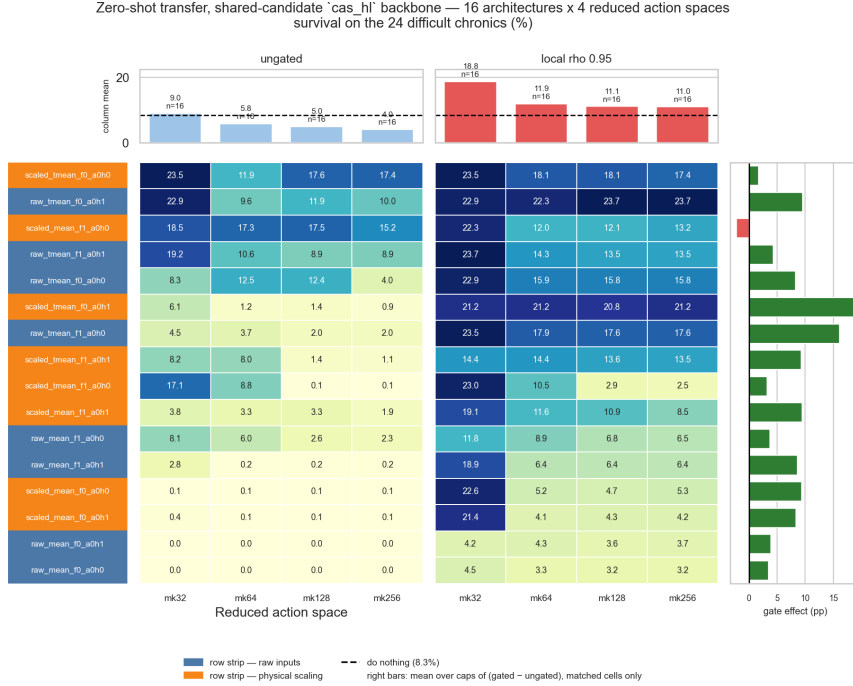
Table A.1: Full-test aggregate results for the sparse-control experiments on **bus14**.

Family	Configuration	Mean survival (%)	Mean action-0
Reference	Do-nothing policy	20.25	1.000
	Baseline flat policy	93.67	0.434
Rho heuristic	Global, $\rho_{safe} = 0.90$	98.36	0.936
	Local, $\rho_{safe} = 0.90$	97.07	0.968
Intervention gate	Final-action MAP	89.82	0.802
	Hierarchical greedy	93.66	0.395
Fixed penalty, flat	$\lambda_{fixed} = 0$	93.67	0.494
	$\lambda_{fixed} = 0.001$	96.75	0.486
	$\lambda_{fixed} = 0.003$	93.97	0.575
	$\lambda_{fixed} = 0.010$	97.44	0.905
Fixed penalty, gated	$\lambda_{fixed} = 0$	78.51	0.787
	$\lambda_{fixed} = 0.001$	73.82	0.862
	$\lambda_{fixed} = 0.003$	94.59	0.887
	$\lambda_{fixed} = 0.010$	85.69	0.974
Adaptive budget	Flat local-safe, $d = 0.10$	98.39	0.979
	Flat local-safe, $d = 0.20$	98.31	0.978
	Flat local-safe, $d = 0.35$	96.61	0.934
	Gated local-safe, $d = 0.20$, hierarchical greedy	33.87	0.999
	Flat plain non-idle, $d = 0.20$	95.08	0.986
Behavioral cloning	Student: balance 0.50, non-idle weight 5, auxiliary weight 0.50	92.14	0.939
	Student: balance 0.20, non-idle weight 3, auxiliary weight 0.25	90.76	0.957

A.2 WCCI Transfer and Target-Control Results

This section gives the detailed results behind Chapters 7–10. Every survival value uses the same 50 WCCI chronics and the easy/difficult split defined in Section 7.2.

A.2.1 Zero-Shot and Intervention Matrices

Figure A.1: Zero-shot difficult-cohort survival across input, **candidate pooling**, descriptor, idle-head and action-space choices.

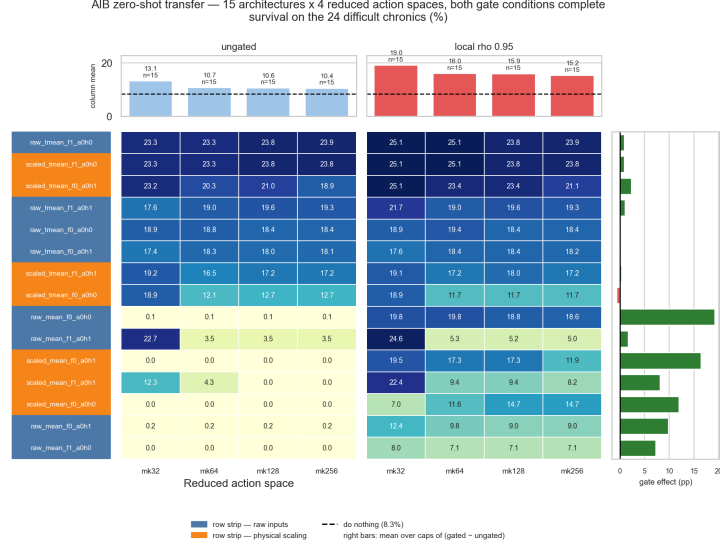


Figure A.2: Difficult-cohort survival with the adaptive intervention budget.

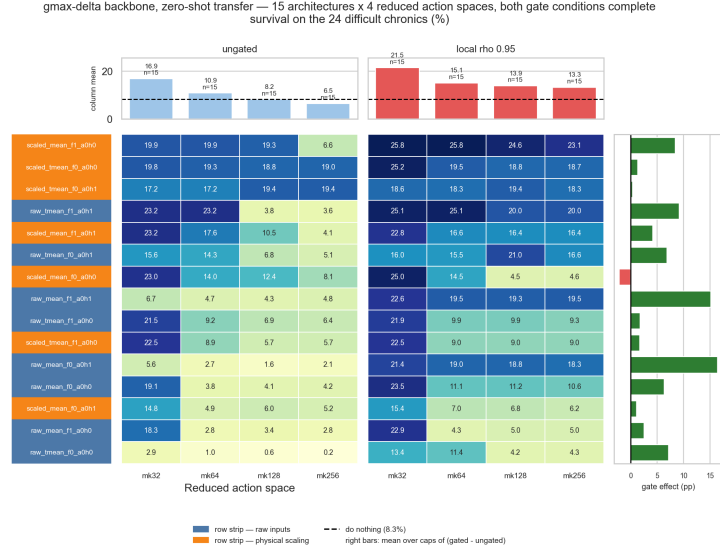


Figure A.3: Difficult-cohort survival with the Gmax-delta action feature.

A.2.2 Target-trained controls

Table A.2: Matched target-trained controls at $k = 64$, using the final checkpoint without an evaluation heuristic. *Easy kept* counts complete episodes among the 26 chronics completed by do nothing.

Family	Architecture	Overall (%)	Difficult (%)	Rescues	Easy kept
Flat MLP	—	32.64	8.82	0	9
Fixed-list GNN	bus_e0n0v0	29.47	4.50	0	4
Fixed-list GNN	bus_e1n0v0	39.63	5.15	0	9
Fixed-list GNN	het_ga2b_1b2a	39.54	4.59	0	10
Fixed-list GNN	het_gbi_1bi	44.95	6.80	0	15

A.2.3 Per-chronic diagnostic

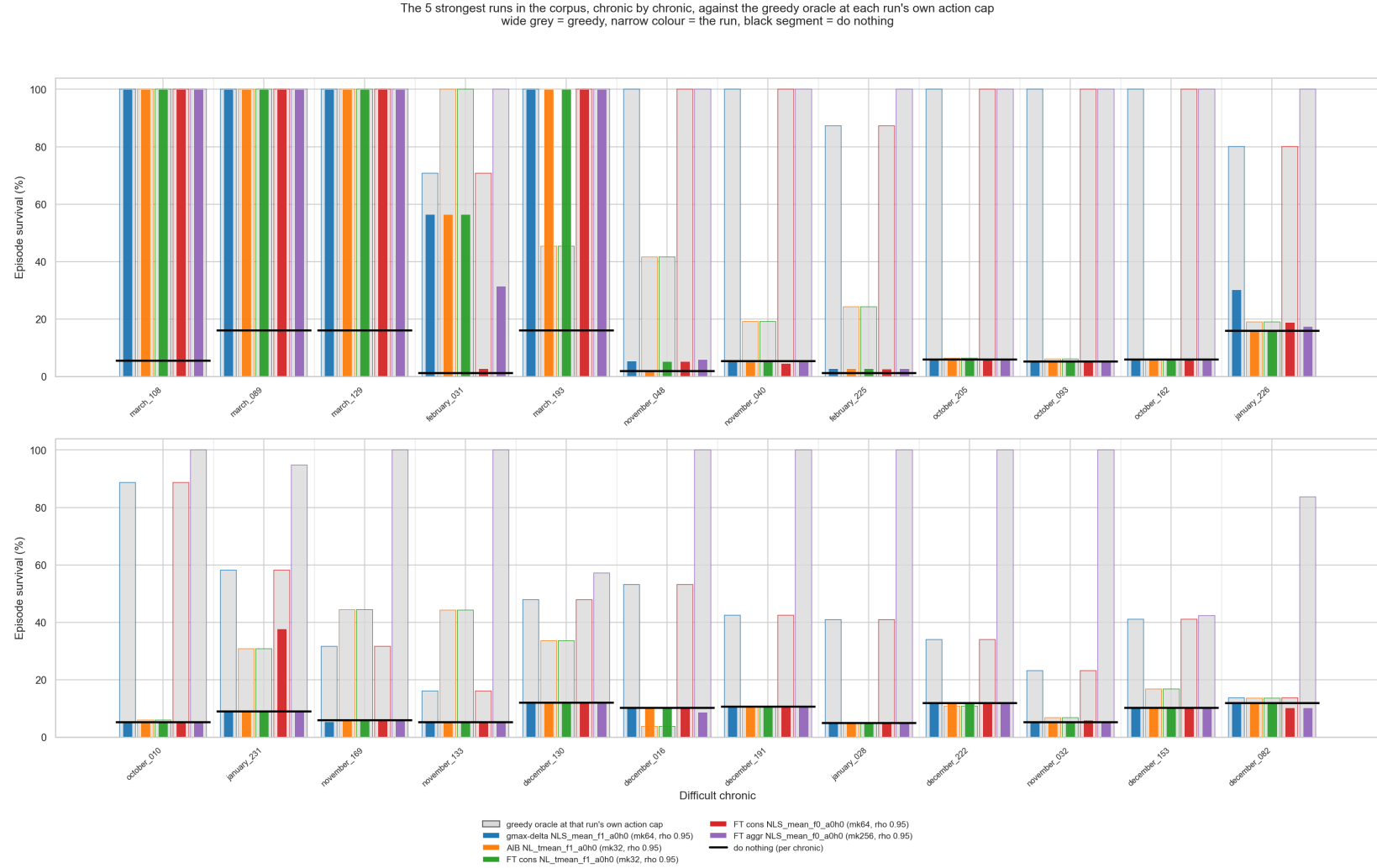


Figure A.4: Per-chronic survival for selected policies on the difficult cohort, with do nothing and the greedy reference.

Appendix B

Graph-Policy Implementation Reference

This appendix keeps only the implementation details needed to reproduce the graph actors: candidate-action metadata, exact input inventories, masking rules, and decentralized information boundaries.

B.1 Candidate-action scorer implementation

A Grid2Op action is mapped once to the graph rows of the line endpoints, generators, loads, and substations that it modifies. All busbars of a modified substation are included so that both the current and destination contexts are available. A boundary action uses only rows already present in the local graph. For example, moving one endpoint of line 4 at substation 6 maps to the two busbar rows of substation 6 and the line-node row of line 4.

Candidates touch different numbers of nodes, so their row lists are padded to one tensor width and a Boolean mask removes the padding during aggregation. The optional descriptor ϕ_j adds five binary operation types and seven counts, normalized within the agent’s action space.

Table B.1: Static candidate-action descriptor ϕ_j .

Block	Entries
Binary indicators	Do nothing; modifies a substation; touches a line, load, or generator
Scaled counts	Modified substations and topology entries; line-status and line-endpoint changes; generator, load, and other changes

B.2 Exact graph inputs

All node types in one representation share a common feature width; unavailable columns are zero. Inactive candidate nodes are entirely zeroed, disconnected, and excluded from readout.

B.2.1 Node inputs

Table B.2: Nonzero node inputs under the default node-angle representation. With **edge_diff**, equipment angles leave the node rows; the busbar and disaggregated schemas place the angle difference on the transmission-line edge, while the line-node schema stores **theta_diff** on the line row.

Representation	Node type	Populated input features
bus	Busbar	gen_p , gen_theta , load_p , load_theta , time_before_cooldown_sub , domain_mask
disaggregated	Busbar Generator Load	is_busbar , time_before_cooldown_sub , domain_mask is_generator , p , theta , domain_mask is_load , p , theta , domain_mask
disaggregated line-node	Busbar Generator/load Transmission line	is_busbar , time_before_cooldown_sub , domain_mask Type indicator, p , theta , domain_mask is_transmission_line , line_status , rho , overflow and cooldown counters, optional maintenance times, domain_mask
Augmented schema	Summary/virtual node	Learned structural vector; no raw physical feature row

B.2.2 Edge inputs

Table B.3: Directed relations and edge inputs. Under **edge_diff**, the transmission-line block of the busbar and disaggregated schemas also contains **theta_diff**.

Representation	Directed relation	Edge inputs
bus	Busbar $\leftrightarrow$ busbar	Transmission-line state; structural relations use an all-zero row
disaggregated	Busbar $\leftrightarrow$ busbar Generator/load $\leftrightarrow$ busbar Same-substation busbars	Transmission-line state plus line-relation indicator Zero physical block plus direction-specific relation relation_same_substation_busbar = 1; other entries zero
disaggregated line-node	Busbar $\leftrightarrow$ line node Generator/load $\leftrightarrow$ busbar Same-substation busbars	Origin/extremity-and-direction relation; line state is stored on the line node Direction-specific relation relation_same_substation_busbar = 1; other entries zero
Augmented schema	Busbar $\leftrightarrow$ summary/virtual	Zero edge vector

B.2.3 Deterministic physical scales

Table B.4: Fixed scales used for continuous graph inputs.

Features	Scale	Source
Power inputs	Largest generator rating	Grid model
Angles and angle differences	180 degrees	Fixed angular scale
Substation/line cooldown	Corresponding maximum	Grid2Op parameter
Overflow counter	Allowed overflow duration	Grid2Op parameter
Maintenance time/duration	Episode horizon	Chronic horizon

B.3 Graph construction and local boundaries

B.3.1 Candidate sizes and masking

Let S , B , L , N_g , and N_d denote the included substations, busbars per substation, transmission lines, generators, and loads. Attachment directions d_g, d_d, d_ℓ equal one or two, and P_ℓ is the number of represented line endpoints.

Table B.5: Candidate graph size before masking. Self edges add $|\mathcal{V}|$ edges and same-substation edges add $SB(B - 1)$.

Schema	Candidate nodes	Candidate directed edges
bus	SB	$2B^2L$
disaggregated	$SB + N_g + N_d$	$2B^2L + B(d_gN_g + d_dN_d)$
disaggregated line-node	$SB + N_g + N_d + L$	$Bd_\ell P_\ell + B(d_gN_g + d_dN_d)$

Candidate rows stay at fixed indices throughout an episode. An inactive edge is zeroed and removed with `edge_mask`; an inactive node and all its incident edges are zeroed, and `node_mask` excludes it from readout. Topology changes therefore update masks without rebuilding tensor shapes.

B.3.2 Substation augmentations

Direct same-substation edges and summary nodes are added only to controlled busbars. The busbar schema represents a direct structural edge with an all-zero edge row; the other schemas activate the typed same-substation relation shown in Table B.3. A summary node uses a learned structural vector and zero-valued attachment edges. Neither augmentation exposes additional contextual equipment.

B.3.3 Decentralized information boundary

Table B.6: Information from another agent’s domain visible to a local actor.

Representation	Neighbor information available
busbar	Shared-line edge and attached far-end busbar, including aggregated generator/load power and angles
disaggregated	Shared-line edge and attached far-end busbar; no neighboring generator/load nodes or private-asset state
disaggregated line-node	Shared-line node only; no neighboring busbar, equipment, or far-end attachment

The centralized critic remains unrestricted during training. Regression tests verify masking, topology-dependent context, and the absence of neighboring private assets from local actor graphs.

Bibliography

- [1] E. B. Fisher, R. P. O'Neill, and M. C. Ferris, "Optimal transmission switching," *IEEE Transactions on Power Systems*, vol. 23, no. 3, pp. 1346–1355, 2008.
- [2] J. Viebahn, M. Naglic, A. Marot, B. Donnot, and S. H. Tindemans, "Potential and challenges of AI-powered decision support for short-term system operations," in *CIGRE Paris Session*, 2022.
- [3] B. Donnot, "Grid2op: A testbed platform to model sequential decision making in power systems," *arXiv preprint arXiv:2011.07929*, 2020.
- [4] A. Marot et al., "Learning to run a power network challenge for training topology controllers," *Electric Power Systems Research*, 2021.
- [5] M. Tan, "Multi-agent reinforcement learning: Independent vs. cooperative agents," in *International Conference on Machine Learning*, 1993.
- [6] R. Lowe, Y. Wu, A. Tamar, J. Harb, P. Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," in *Advances in Neural Information Processing Systems*, 2017.
- [7] C. Yu et al., "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in Neural Information Processing Systems*, 2022.
- [8] M. Hassouna, C. Holzhüter, P. Lytaev, J. Thomas, B. Sick, and C. Scholz, *Graph reinforcement learning for power grids: A comprehensive survey*, 2024. arXiv: [2407.04522](https://arxiv.org/abs/2407.04522). [Online]. Available: <https://arxiv.org/abs/2407.04522>
- [9] M. Subramanian, J. Viebahn, S. H. Tindemans, B. Donnot, and A. Marot, "Exploring grid topology reconfiguration using a simple deep reinforcement learning approach," in *IEEE Madrid PowerTech*, 2021.
- [10] D. Yoon, S. Hong, B.-J. Lee, and K.-E. Kim, "Winning the L2RPN challenge: Power grid management via semi-Markov afterstate actor-critic," in *International Conference on Learning Representations*, 2021.
- [11] A. Chauhan, M. Baranwal, and A. Basumatary, "PowRL: A reinforcement learning framework for robust management of power networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, 2023, pp. 14 757–14 764.
- [12] M. Lehna, J. Viebahn, A. Marot, S. Tomforde, and C. Scholz, "Managing power grids through topology actions: A comparative study between advanced rule-based and reinforcement learning agents," *Energy and AI*, vol. 14, p. 100 276, 2023.
- [13] B. Manczak, J. Viebahn, and H. van Hoof, *Hierarchical reinforcement learning for power network topology control*, 2023. arXiv: [2311.02129](https://arxiv.org/abs/2311.02129). [Online]. Available: <https://arxiv.org/abs/2311.02129>
- [14] E. van der Sar, A. Zocca, and S. Bhulai, *Multi-agent reinforcement learning for power grid topology optimization*, 2023. arXiv: [2310.02605](https://arxiv.org/abs/2310.02605). [Online]. Available: <https://arxiv.org/abs/2310.02605>
- [15] B. de Mol, D. Barbieri, J. Viebahn, and D. Grossi, "Centrally coordinated multi-agent reinforcement learning for power grid topology control," in *Proceedings of the 16th ACM International Conference on Future and Sustainable Energy Systems (e-Energy)*, 2025.
- [16] E. Marchesini et al., "MARL2Grid-TR: A multi-agent RL benchmark in power grid operations," in *International Conference on Learning Representations*, 2026.
- [17] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," *International Conference on Learning Representations*, 2017.

- [18] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, “Graph attention networks,” in *International Conference on Learning Representations*, 2018.
- [19] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, “How powerful are graph neural networks?” *International Conference on Learning Representations*, 2019.
- [20] M. de Jong, J. Viebahn, and Y. Shapovalova, *Generalizable graph neural networks for robust power grid topology control*, 2025. arXiv: [2501.07186](https://arxiv.org/abs/2501.07186). [Online]. Available: <https://arxiv.org/abs/2501.07186>
- [21] E. Marchesini et al., *RL2Grid: Benchmarking reinforcement learning in power grid operations*, 2025. arXiv: [2503.23101](https://arxiv.org/abs/2503.23101). [Online]. Available: <https://arxiv.org/abs/2503.23101>
- [22] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [23] J. Achiam, D. Held, A. Tamar, and P. Abbeel, “Constrained policy optimization,” *International Conference on Machine Learning*, 2017. [Online]. Available: <https://arxiv.org/abs/1705.10528>
- [24] A. Stooke, J. Achiam, and P. Abbeel, “Responsive safety in reinforcement learning by PID lagrangian methods,” *International Conference on Machine Learning*, 2020. [Online]. Available: <https://arxiv.org/abs/2007.03964>
- [25] N. Carrara, E. Leurent, R. Laroche, T. Urvoy, and O.-A. Maillard, “Budgeted reinforcement learning in continuous state space,” *Advances in Neural Information Processing Systems*, 2019. [Online]. Available: <https://arxiv.org/abs/1903.01004>
- [26] L. Lautenbacher et al., *Multi-objective reinforcement learning for power grid topology control*, 2025. arXiv: [2502.00040](https://arxiv.org/abs/2502.00040). [Online]. Available: <https://arxiv.org/abs/2502.00040>
- [27] W. Hu et al., “Strategies for pre-training graph neural networks,” in *International Conference on Learning Representations*, 2020.